applied sciences

MDPI

Article

Class Incremental Deep Learning: A Computational Scheme to Avoid Catastrophic Forgetting in Domain Generation Algorithm Multiclass Classification

João Rafael Gregório *,†, Adriano Mauro Cansian *,† and Leandro Alves Neves *,†

Department of Computer Science and Statistics (DCCE), São Paulo State University (UNESP), São José do Rio Preto, São Paulo 15054-000, Brazil

* Correspondence: rafael.gregorio@unesp.br (J.R.G.); adriano.cansian@unesp.br (A.M.C.); leandro.neves@unesp.br (L.A.N.)

† These authors contributed equally to this work.

Abstract: Domain Generation Algorithms (DGAs) are algorithms present in most malware used by botnets and advanced persistent threats. These algorithms dynamically generate domain names to maintain and obfuscate communication between the infected device and the attacker's command and control server. Since DGAs are used by many threats, it is extremely important to classify a given DGA according to the threat it is related to. In addition, as new threats emerge daily, classifier models tend to become obsolete over time. Deep neural networks tend to lose their classification ability when retrained with a dataset that is significantly different from the initial one, a phenomenon known as catastrophic forgetting. This work presents a computational scheme composed of a deep learning model based on CNN and natural language processing and an incremental learning technique for class increment through transfer learning to classify 60 DGA families and include a new family to the classifier model, training the model incrementally using some examples from known families, avoiding catastrophic forgetting and maintaining metric levels. The proposed methodology achieved an average precision of 86.75%, an average recall of 83.06%, and an average F1 score of 83.78% with the full dataset, and suffered minimal losses when applying the class increment.

Keywords: incremental learning; multiclass classification; deep learning; DGA; botnets; cybersecurity

check for updates

Citation: Gregório, J.R.; Cansian, A.M.; Neves, L.A. Class Incremental Deep Learning: A Computational Scheme to Avoid Catastrophic Forgetting in Domain Generation Algorithm Multiclass Classification. *Appl. Sci.* **2024**, *14*, 7244. https://doi.org/10.3390/app14167244

Academic Editors: George Drosatos, Konstantinos Rantos and Konstantinos Demertzis

Received: 28 June 2024
Revised: 4 August 2024
Accepted: 7 August 2024
Published: 17 August 2024

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Botnets consist of groups of computers or devices infected with malware, which can be remotely controlled via a command and control (C2) server [1]. Attackers leverage botnets for various cyberattacks, such as distributed denial of service (DDoS) attacks, directing infected devices to target specific systems, or systematically extracting information from corporate and government networks [2]. Botnets use Domain Generating Algorithms (DGAs) as a strategy to maintain and obfuscate communication between a botclient and C2 servers. DGAs are codes embedded in most malware families that generate domain names. These domains are used to maintain communication between the infected machines and the C2 servers [3,4]. This approach allows the C2 servers to change their IP addresses without disrupting communication with the botnet. Consequently, DGAs enhance the resilience of communication between C2 servers and botnet members and improve the obfuscation of the C2 server's location [1].

Attackers register numerous domains that are compatible with their DGAs. This strategy ensures that if one domain is block-listed for abuse, another can take its place. The vast number of DGA-generated domains makes using block lists impractical, as maintaining a list of thousands of domains can significantly hinder firewall performance [5].

One of the most significant cyberattacks that used DGA at some point in its lifecycle was the attack on SolarWinds [6] that occurred in 2020. This gained notoriety when hackers

managed to gain unauthorized access to the source code of the Orion system, which is used for monitoring technological infrastructure. More than 18,000 SolarWinds customers installed versions infected with the Sunburst malware, including technology giants such as Microsoft, Intel, and Cisco. The Sunburst malware used an elaborate DGA mechanism [3] that, in addition to maintaining communication between the infected station and the C2 servers, was also capable of accurately identifying which infected station the information obtained came from. Therefore, identifying requests for DGA domains on a network or detecting them during registration can mitigate botnet-related threats, prevent potential attacks on corporate networks, and support proactive cybersecurity measures.

Approaches using deep neural networks have been used by researchers around the world to solve complex problems in a wide range of fields, from electrical engineering [7], to natural language processing and sentiment classification [8], and also in the area of cybersecurity.

In this context, numerous efforts have been made to detect these DGAs through automated methods utilizing artificial intelligence, with the most recent approaches typically employing deep learning techniques. Ref. [9] use the NetLab360 dataset, which encompasses approximately 35 DGA families, for their analysis of DGA examples. They encode the input characters into values ranging from 0 to 38 to derive the initial feature vector. A Convolutional Neural Network (CNN) model, concatenated with a BiLSTM network, is proposed to extract meta-features for DGA detection. The study reports an accuracy of 94.51%, a recall of 95.05%, and a precision of 93.11% for binary detection. In [4], researchers utilize pre-trained word embedding models, including BERT [10] and ELMo [11], to generate the initial feature vector from domain names. This initial vector is input into a CNN and then processed by a classifier, achieving a peak accuracy of 96.08%. Ref. [12] presents a model based on CNN and character-level embeddings, utilizing domain names converted to ASCII as input. Using the Majestic Million [13] dataset for legitimate domain examples and the NetLab360 [14] dataset for examples from 56 threat families, the model achieves an accuracy of 99.12% and a precision of 99.33%.

Considering the multitude of distinct threats, some recent efforts have concentrated on addressing the problem of classifying DGAs into families to ascertain the specific threat associated with a given DGA. In [15], the proposal was a hybrid model using transformers and CNN to classify around 50 DGA families in the NetLab360 dataset. The obtained results presented are a macro precision of 71.66% and a macro F1 score of 67.44%. Another work, Ref. [16], presents a study on multiclass classification involving 12 categories, comprising 11 word-based DGA families and legitimate examples. The proposed model utilizes a Bi-LSTM deep neural network, with input data derived from word embeddings. The study reports an F1 score of 94.40% for DGAs and 77.30% for the legitimate class. Therefore, recent DGA detection models exhibit performance metrics nearing ideal results, with accuracy rates surpassing 95%. The researchers in [17] address multiclass DGA classification and the problem of class imbalance using an approach based on a deep learning model and a transfer learning technique. The work addresses the issue of catastrophic forgetting in transfer learning between deep neural network models to mitigate the class imbalance problem.

However, recent studies focusing on the classification of these DGAs suggest that there is still room for improvement, especially with regard to the increase in classes. It can also be seen that all works start from a well-defined dataset, with a specific number of DGA families, and use the entire dataset to train and test their models.

Given the daily emergence of new threats and the resultant new DGA families, it would be highly valuable to incrementally retrain [18] a model that already exhibits adequate performance metrics. This approach should allow the model to identify new families without compromising its ability to classify previously known families and without requiring the use of all examples from all known families [19]. However, in artificial neural networks, there is a phenomenon known as catastrophic forgetting [20,21]. This occurs when a pre-trained model is retrained with a dataset different from the one originally used,

leading to an abrupt update of all network parameters. Consequently, this results in a significant loss of the model's prior classification capabilities.

In this study, a computational scheme is presented consisting of a deep learning model based on CNN and character-level embeddings for classifying DGAs into 61 known families and an incremental learning technique for class increment, allowing the pre-trained model to classify a new DGA family without losing the ability to classify previously known families. The proposed methodology was defined using transfer learning, incremental training, and a strategy for selecting examples for incremental training. This approach effectively mitigates the phenomenon of catastrophic forgetting. The main contributions of this article are as follows:

- Presents a computational scheme for DGA multiclass classification, with class increment, based on a deep learning model and a transfer learning class increment technique.
- Proposes a deep learning model based on CNN and character-level embedding for multiclass classification of DGAs.
- Demonstrates a technique for class increment for the proposed classifier model, using a few examples of known classes and avoiding catastrophic forgetting.

This work is organized as follows: Section 2 details the materials and methodology employed in data pre-processing, describes the datasets used, and describes the tools and technologies that supported this work, along with a description of the proposed model. Also in Section 2, the technique for class increment, avoiding catastrophic forgetting, is detailed. Section 3 presents the results obtained by the model in several tests, comparisons of different configurations of the proposed model, and results in relation to the existing literature in the classification of 61 DGA families. The results obtained with the application of the proposed technique for adding a new class to the previously trained model are also presented. Finally, Section 4 offers a discussion, conclusions, and proposals for future works.

2. Materials and Methods

This section outlines the methodology utilized for the development of the proposed model, including the pre-processing of datasets and the technologies employed. Section 2.1 describes the development environment and applied technologies, details the datasets and their pre-processing, and explains the division into training and testing sets. Section 2.2 elaborates on the proposed model, detailing each layer along with its functions and hyperparameters, and providing a summary to facilitate the reproduction of the experiment. Section 2.3 introduces the proposed class incremental learning technique, and Section 2.4 discusses the metrics used to evaluate the results obtained from training and testing the proposed model.

2.1. Development Environment and Datasets

To develop this work, we used a notebook Dell, acquired in Brazil, with 32 GB of RAM, 11th Generation Intel Core i5 processor with 12 processing cores, RTX 3050 GPU with 4 GB of graphics processing memory, 1 TB of storage in NVMe units, and 4 TB of storage in an external hard drive for processing data and building models, training algorithms, and analyzing results. All algorithms were developed using the Python 3.8 programming language with the help of the Pandas [22] and NumPy [23] libraries for manipulating the datasets. The Scikit-learn [24] library was used to present the model metrics and the processing stages of the datasets. The deep learning models were developed using the open-source TensorFlow 2.8 [25] library developed and maintained by Google. The notebooks containing the complete experiment have been published, thus allowing the complete reproduction of this work. The source code is available in the Data Availability Statement.

For the training and testing of the models proposed in this work, the DGArchive database maintained by the Fraunhofer Institute for Communication, Information Processing, and Ergonomics (FKIE) [26], a German institution focused on the development of technologies for the early detection and mitigation of risks, has been used as a source of

algorithm-generated domain examples. The institute kindly provided us with access to its API, allowing us to query both historical DGA data and daily cataloged examples. Through the API, it collected examples of DGAs observed over a 15-day period, from 1 January to 15 January 2024, encompassing 122 DGA families. For the development of this work, only families with at least 1000 examples in the dataset were selected, reducing the number of families to 61. Table 1 presents the distribution of examples for each DGA family that make up the dataset used.

Table 1. Distribution of examples by DGA family in the dataset.

#	Family	Examples	#	Family	Examples	#	Family	Examples
1	gameover	165,000	22	pushdo	10,468	43	padcrypt	2520
2	virut	149,942	23	suppobox	10,350	44	shifu	2331
3	murofet	113,400	24	locky	10,320	45	pseudoman	2300
4	tinba	111,368	25	pykspa2s	9960	46	ebury	2000
5	necurs	107,328	26	zloader	9109	47	fobber	2000
6	metastealer	80,000	27	conficker	7500	48	proslikefan	1950
7	qakbot	60,000	28	darkwatchman	7500	49	tinynuke	1920
8	flubot	55,577	29	pushdotid	6000	50	sphinx	1920
9	bazarloader	46,924	30	qadars	6000	51	rovnix	1900
10	monerominer	37,430	31	ramdo	6000	52	gozi	1757
11	ranbyus	34,300	32	corebot	5850	53	nymaim2	1600
12	banjori	32,120	33	nymaim	5669	54	bamital	1560
13	urlzone	32,020	34	necro	4757	55	pitou	1464
14	tinyfluff	30,000	35	xxhex	4400	56	orchard	1447
15	qsnatch	27,787	36	symmi	4320	57	phorpiex	1431
16	ramnit	20,109	37	gameover_p2p	3000	58	dircrypt	1400
17	simda	16,474	38	murofetweekly	3000	59	mydoom	1226
18	tufik	15,300	39	ngioweb	2924	60	grandoreiro	1095
18	dyre	15,000	40	emotet	2880	61	darkshell	1049
20	cryptolocker	15,000	41	vawtrak	2700			
21	chinad	15,000	42	m0yvtdd	2664			

Domain names were encoded character by character to ASCII code. The maximum length checked across the domain name samples was 70 characters, so smaller domain names were padded with zeros from the end to the maximum length. No data cleaning was necessary because the data received by the above-cited API was already tabulated, and the only input datum used was the domain name. No normalization or transformation was applied to the data to keep the model's input data as close to reality as possible.

To train and test the proposed model, as well as the other models to compare the results, the dataset was separated in the proportion of 70% for training and 30% for testing into three different random states using the repeated holdout [27] principle to mitigate the impact of selection on results. For the class increment experiment, a family was selected and removed from the dataset; the remaining dataset was then separated into a proportion of 70% for training and 30% for testing. In this case, three families were chosen depending on the number of examples present in the dataset, namely Metastealer with 80,000 examples in the dataset, Suppobox with 10,350 examples in the dataset, and Darkshell with only 1049 examples available.

2.2. Proposed Model

The proposed deep learning model, a fundamental part of the proposed computational scheme, is based on CNN and natural language-processing techniques, with its data input into a character-level embedding layer [12,28]. This receives an initial vector composed of 70 ASCII values relative to each character in the domain to be classified. The embedding layer then converts each value into a vector of 60 values, related to the context of that character in the domain name in question. The output of the embedding layer is then connected to a CNN structure that seeks to extract meta-features relevant to domain name

classification from the 70 vectors of 60 values provided by the next higher layer. This CNN structure has six convolutional layers, in addition to relu and maxpooling layers. The output of the CNN structure is then connected to a flattened layer to adjust the shape of the data and then deliver it to a dense classifier. The last dense layer of the classification has a number of neurons equal to the number of classes for which the model is being trained and uses softmax as an activation function. This function assigns a floating point value to each output neuron, and the sum of these values will always be equal to 1 and the highest observed value represents the class to which the model classified the input example. Figure 1 presents a conceptual model of the proposed classifier model; the model itself is available in full in the Kaggle Notebook mentioned in Section 1.

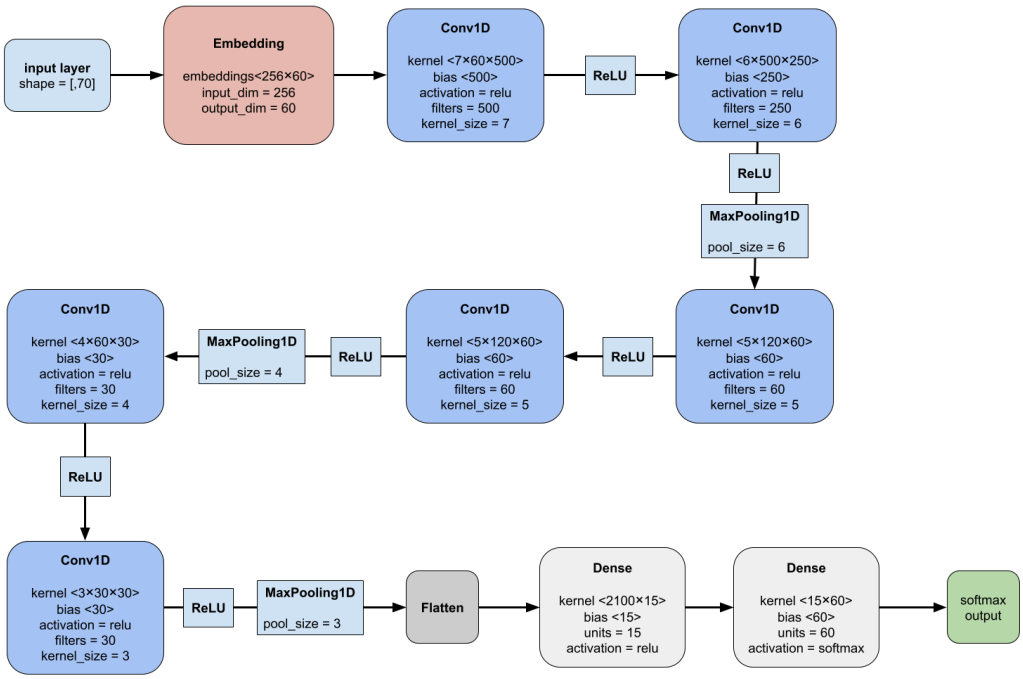


Figure 1. Proposed model concept.

2.2.1. Embedding Layer

Embedding layers are extensively utilized in natural language processing (NLP), where each word is represented by an array of floating point values [29,30]. These layers are trainable, enabling the network to learn the representation of each word. In this study, character-level embeddings are employed. Consequently, each character of a domain name, after being converted to its ASCII code, is processed by the embedding layer, where it is represented by a vector of 60 values. This feature vector, constructed by the embedding layer, is subsequently passed to the CNN component of the proposed model.

2.2.2. Deep Convolutional Neural Network

Convolutional neural networks (CNNs) can identify specific features in images with minimal pre-processing requirements [29,30]. In this work, the CNN component of the proposed model is designed to extract relevant features for the analyzed classes from the feature vector generated by the embedding layer at the model's input. The model comprises six convolutional layers of varying dimensions, forming a deep CNN architecture.

2.2.3. Flatten and Dense Layers

The flattened layer was used after the convolutional component of the proposed model to reshape the feature vector generated by the previous layers, ensuring compatibility with the fully connected dense layers positioned at the end of the model. Finally, two fully connected layers, with 15 and n neurons, respectively, with n being the number of classes

for which the model is being trained, serve as classifiers using the feature vector received from the flattened layer. The last dense layer uses softmax (Equation (1)) as the activation function; this returns values between 0 and 1 for each of the output neurons, whose sum will always be equal to 1, and the largest value is considered the class for which the model classified the example.

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)} \quad (1)$$

2.2.4. Hyperparameters and Model Concept

Another crucial aspect of the proposed model is the selection of hyperparameters. In deep learning models, hyperparameters are not learned during training but must be defined beforehand [31,32]. These parameters are set externally to the model and play a pivotal role in shaping the architecture and training process of the neural network. Unlike model parameters such as weights and biases, which are automatically adjusted by the optimization algorithm during training, hyperparameters are predetermined and have a direct impact on the model's performance and behavior. The hyperparameters that were used to compile the proposed model are detailed in Table 2.

Table 2. Hyperparameters used to compile the proposed model.

Hyperparameter	Value
L1 Regularizer 1×10^{-5}	
L2 Regularizer	1×10^{-4}
Optimizer	Adam
Activation Function	ReLU
Learning Rate	1×10^{-4}
Batch Size	250
Loss Function	Categorical Crossentropy

The L1 and L2 regularizers were employed to apply penalties to the layer parameters, which are added to the loss function to prevent overfitting [33]. The chosen values, 1×10^{-5} and 1×10^{-4}, respectively, are sufficiently low to avoid a sudden impact on the training process. The Adam optimizer was selected due to its computational efficiency as a stochastic gradient descent method, particularly suitable for problems with a large number of parameters and data [34]. The ReLU activation function was chosen to increase the sparsity of the network by converting negative values to zero [30]. Additionally, the network's learning rate was set to a low value, 1×10^{-4}, to facilitate slower parameter updates and thereby aim for the best global minimum loss.

For an initial evaluation of the proposed model for classifying 61 DGA families, the dataset was separated in the proportion of 70% for training and 30% for testing three times, with different random states, for 10% validation of the training set was selectively selected with the validation_split function. For each selection, the model was trained for 20 epochs, the best validation accuracy was saved using the callback function, and then the test set prediction was performed. The results obtained in this initial experiment are presented in Section 3.

2.3. Class Incremental Learning Technique

This experiment was carried out in two stages. In the first stage, the examples of 60 of the 61 families present in the dataset were separated, reserving examples of one family to perform the class increment. The goal is to have a trained multiclass classifier presenting good metrics in the classification among the 60 DGA families and then, using a reduced number of examples of classes already known by the model, add a new class to the classifier, maintaining an adequate level of metrics.

The remaining group, with 60 families, was divided into proportions of 70% for training and 30% for testing. The proposed model was then trained by 20 epochs and the metrics were evaluated with the test set. For incremental training purposes, training set prediction has been run and 500 examples from each family that the model classified correctly were selected. This reduced set of examples represents about 3% of the initial training dataset and will be used to avoid catastrophic forgetting and guarantee the increment of the new class without the model losing its ability to classify known classes.

The trained multiclass model was then changed in its last layer, removing the last dense layer with 60 neurons and adding a dense layer with 61 neurons. From this moment on, incremental training occurs in two stages, in the first stage all layers of the model, except the last one of 61 neurons, are defined as non-trainable, and thus only the weights of the last layer will be updated, maintaining the weights of feature extractor layers without updating. The model is then trained for 20 epochs with the incremental training set, with 500 examples from each known class that were correctly classified by the model plus 70% of the examples from the new class. The test suite also receives the addition of 30% of the examples of the new class. After this incremental training step, all layers of the model were defined as trainable, and the learning rate set to 1×10^{-5}, training the model one last time for another 20 epochs and performing the prediction on the test set. The complete process of the incremental learning technique through transfer learning is shown in Figure 2.

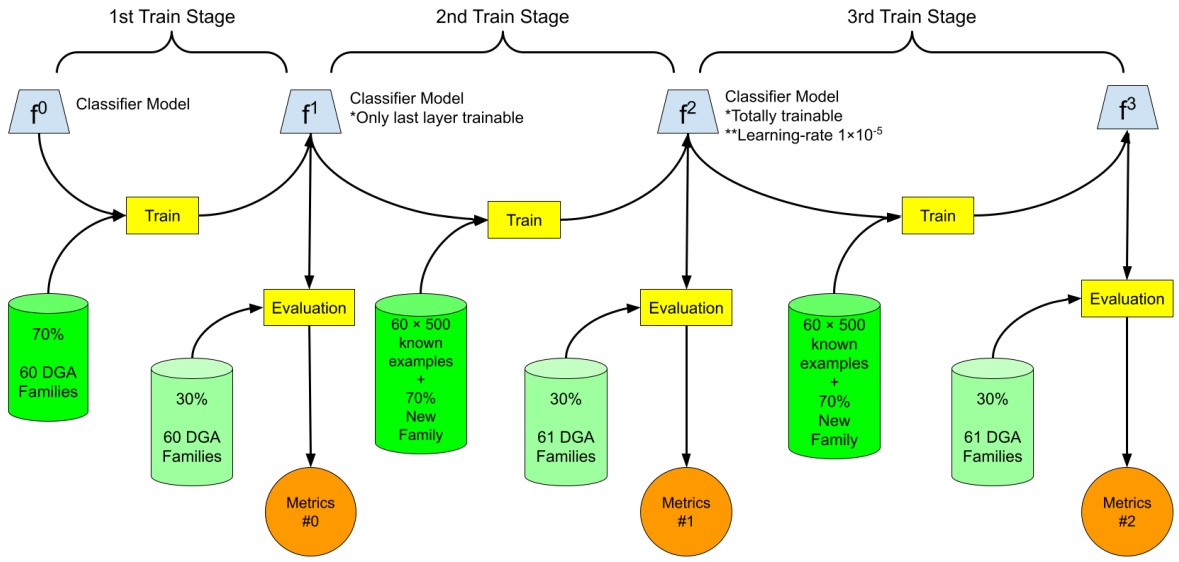


Figure 2. Proposed class incremental learning by transfer learning process.

In the proposed methodology, stage f0 represents the classifier model before initial training. Stage f1 therefore represents the classifier model already trained to classify among 60 DGA families. In stage f2, the model has been trained incrementally, with 500 examples of each of the 60 families known by the model that were correctly classified added to 70% of the examples of the new family. At this stage, only the last layer of the model is trainable; all other layers are defined as non-trainable, and this is the first stage of incremental training. In stage f3, the classifier model was trained again, with all layers defined as trainable and a learning rate 10 times lower, to smoothly update the parameters of all layers. At all stages after each training, the metrics were checked to verify the effectiveness of the proposed methodology. The results obtained in the proposed class increment process performed with three different families are presented in Section 3.

2.4. Metrics and Evaluations

To evaluate the model, the most commonly accepted metrics for evaluating deep learning models was used: Precision (Prec.) (Equation (2)), Recall (Rec.) (Equation (3)), F1 score (F1) (Equation (4)). These metrics are obtained as follows:

$$Precision(Prec.) = \frac{TP}{FP + TP} \tag{2}$$

$$Recall(Rec.) = \frac{TP}{FN + TP} \tag{3}$$

$$F1score(F1) = \frac{2 \times TP}{2 \times TP + FP + FN} \tag{4}$$

where

- True Positive (TP): DGA classified correctly in its class;
- False Positive (FP): DGA from another class misclassified in a given class;
- False Negative (FN): DGA of one class misclassified into another class.

To evaluate the effectiveness of the proposed model and the application of NLP techniques to domain names, an additional model derived from the main model, which will be referred to as Model 1, was tested on the same dataset. One model without the embedding layer in the input will be referred to as Model 2. The basic structure of the two tested models is presented in Table 3. For better visualization, the Figure graphically presents the results obtained by the models compared in each of the metrics side by side. Additionally, five other models from recent works were trained and tested on the same dataset: the Bilbo model presented in [35], a model using the pre-trained transformer DistilBERT, a lighter and faster version of the BERT transformer proposed in [36], as well as simple CNN, CNN+LSTM, and RNN models proposed in [28].

The structures of the models tested for comparison were obtained from the GitHub repositories of each author, cited as supplementary material in their works. This direct comparison between the results is important to demonstrate the robustness of the proposed model and its alignment with the state-of-the-art research developed in this regard. Table 3 presents the basic structure of each of the works and models compared on the same training and test dataset.

Table 3. Tested models and their basic structure.

Model	Structure
Model 1 (Proposed)	CNN+Embedding+ReLU+MaxPooling
Model 2 (Proposed)	CNN+ReLU+MaxPooling
Bilbo [35]	Embedding+CNN+LSTM
DistilBERT [36]	DistilBERT+DNN
CNN [28]	CNN Simple
CNN+LSTM [28]	CNN+LSTM Simple
RNN [28]	RNN Simple

To evaluate the effectiveness of applying the proposed class increment technique, Model 1 was subjected to training to classify among 60 DGA families and then a new family was added to the model using the proposed technique, and metrics in each of the training steps were noted. The metrics of the same Model 1 trained exclusively with the reduced dataset were also noted to compare results. This evaluation was repeated three times using three different DGA families as incremental classes, with different numbers of examples in the dataset, namely Metastealer with 80,000 examples, Suppobox with 10,350 examples, and Darkshell with 1049 examples in the dataset.

3. Results and Discussion

This section presents the results obtained by the two-step DGA classifier model. First, it was evaluated that the metrics obtained by the model trained with 70% of the examples contained in the dataset and tested with 30% of the examples. To make it possible to evaluate the impact of character-level embedding on the metrics obtained, the same tests were carried out with an altered version of the proposed model, called Model 1, removing the embedding layer. This model without the embedding layer is called Model 2. Also, to assess the model's performance in relation to the state-of-the-art DGA classification efforts, five other models available in recent works were implemented, being trained and tested on the same dataset. In a second moment, Model 1 was subjected to the incremental training process to increase the class, as presented in Section 2.3.

3.1. Classic Train Evaluation

All models presented were trained and tested three times with random selections of data from the same dataset containing 61 DGA families in the order of 70% for training and 30% for testing. The precision, recall, and F1 score of each of the 61 families were recorded for each training and test. Table 4 presents the general average of the metrics among all DGA families during the tests carried out for each of the tested models.

Table 4. Average of the metrics obtained by Model 1, Model 2, and other models in direct comparison to classifying 61 DGA families.

Model	Avg. Prec.	Avg. Rec.	Avg. F1
Model 1 (Proposed)	86.75%	83.06%	83.78%
Model 2 (Proposed)	79.78%	72.39%	72.87%
Bilbo [35]	64.39%	58.22%	58.22%
DistilBERT [36]	53.71%	36.65%	38.05%
CNN [28]	77.06%	68.38%	69.68%
CNN+LSTM [28]	73.83%	70.28%	70.19%
RNN [28]	47.33%	44.11%	43.69%

It is possible to observe that Model 1 obtained significantly higher metrics than the other models tested; on average, Model 1 presented a precision 20.73% higher than the average of the other models and 6.97% higher than the second best, which is Model 2; recall 24.72% higher than the average of the other models and 10.67% higher than the recall obtained by Model 2; and an F1 score 25% higher than the average of the other models, and 10.91% higher than the F1 score obtained by Model 2. It can also be seen that models based on convolutional neural networks obtained better metrics than the others. Therefore, in addition to obtaining metrics superior to the other models on the same set of data, Model 1 obtained, on average, metrics 9.52% higher than those obtained by Model 2, a model that did not use character-level embedding in the input data. For better visualization, Figure 3 graphically presents the results obtained by the models compared in each of the metrics side by side.

A more detailed overview with results by family is presented in Appendix A. Table A1 presents the results obtained by Model 1 in the classification among the 61 DGA families. Table A2 presents the results obtained by Model 2 in classifying the 61 DGA families. Table A3 presents the results obtained by the Bilbo model in classifying the 61 DGA families. Table A4 presents the results obtained by the DistilBERT model in classifying each of the 61 DGA families. Table A5 presents the results per family obtained by the CNN model. Tables A6 and A7 present the results obtained by the CNN+LSTM and RNN models, respectively, in classifying the 61 DGA families.

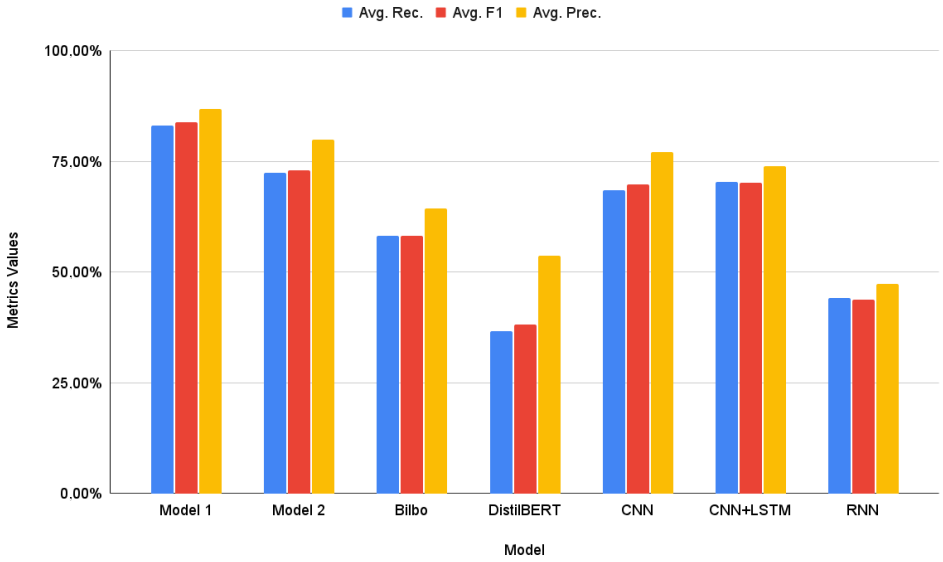


Figure 3. Graphical comparison of the metrics obtained by the tested models.

3.2. Class Incremental Learning Method

Once Model 1 proved to be robust, with metrics in line with the state-of-the-art, even surpassing metrics from published models, it began to verify the effectiveness of the proposed technique for class increment through transfer learning presented in Section 2.3. In this experiment, it was separated by a DGA family so that it could be used as an incremental class; that is, it would be included in the scope of classes known by the model already trained to classify the other classes. For this, it selected three DGA families, with different amounts of samples present in the dataset, namely Metastealer with 80,000 examples, Suppobox with 10,350 examples, and Darkshell with 1049 examples in the dataset. Three incremental training tests were carried out to increase the class using the proposed technique, and in each of the tests, a family was used as the incremental class.

For each of the tests, the averages of the general metrics were calculated at each of the training moments. In the initial stage, the metrics refer to the classifier trained to classify among 60 families. The first stage of incremental training consists of defining all layers of the model as not-trainable except the last one, editing the last layer by adding one more neuron so that the model can classify between 61 classes. Then, the classifier model was retrained with 500 examples of each of the known families that were correctly classified by the classifier in their previous state, along with 70% of the examples of the new family.

In the second stage of incremental training, the model has its layers defined as trainable again and training occurs with a learning rate of 1×10^{-5}, 10 times lower than the training rate used in the initial training, to perform fine adjustments to the weights of the model for the classification of the 61 classes. To verify the effectiveness of incremental training and assess whether there was indeed a transfer of learning in incremental training, it was trained an identical model, from scratch, only with the reduced set of examples used in the two stages of incremental training. The average precision, recall, and F1 score among all families classified by the model in the initial stage, in each of the incremental training stages, and in the control training for each of the families used as an incremental class are presented in Tables 5–7.

Table 5 presents the general averages of precision, recall, and F1 score obtained when using the Metastealer class as an incremental class, for each of the incremental training steps of Model 1. The metrics extracted by each family are presented in Appendix A, Table A8 presents the metrics obtained by Model 1 for the initial training without the Metastealer family, Table A9 presents the results obtained by Model 1 after the first stage of incremental training, with examples of the new Metastealer class. Table A10 presents the results after

the second training stage, and Table A11 shows the results of Model 1 trained exclusively on the reduced dataset.

Table 5. Average of the metrics obtained by the proposed Model 1 during the incremental process using Metastealer as an incremental class.

Train Stage	Avg. Prec.	Avg. Rec.	Avg. F1
Initial	86.53%	80.54%	81.76%
1st Incremental	80.86%	84.21%	81.04%
2nd Incremental	81.27%	84.33%	81.27%
From Scratch	59.19%	73.96%	62.50%

Table 6 presents the general averages of precision, recall, and F1 score obtained when using the Suppobox class as an incremental class, for each of the incremental training steps of Model 1. The metrics extracted by each family are presented in Appendix A, Table A12 presents the metrics obtained by Model 1 for the initial training without the Metastealer family, Table A13 presents the results obtained by Model 1 after the first stage of incremental training, with examples of the new Metastealer class, Table A14 presents the results after the second training stage, and Table A15 shows the results of Model 1 trained exclusively on the reduced dataset.

Table 6. Average of the metrics obtained by the proposed Model 1 during the incremental process using Suppobox as an incremental class.

Train Stage	Avg. Prec.	Avg. Rec.	Avg. F1
Initial	86.72%	82.53%	82.79%
1st Incremental	78.23%	79.24%	76.03%
2nd Incremental	82.00%	84.88%	81.51%
From Scratch	57.16%	73.23%	60.38%

Table 7 presents the general averages of precision, recall, and F1 score obtained when using the Darkshell class as an incremental class, for each of the incremental training steps of Model 1. The metrics extracted by each family are presented in Appendix A, Table A16 presents the metrics obtained by Model 1 for the initial training without the Metastealer family, Table A17 presents the results obtained by Model 1 after the first stage of incremental training, with examples of the new Metastealer class, Table A18 presents the results after the second training stage, and Table A19 shows the results of Model 1 trained exclusively on the reduced dataset.

Table 7. Average of the metrics obtained by the proposed Model 1 during the incremental process using Darkshell as an incremental class.

Train Stage	Avg. Prec.	Avg. Rec.	Avg. F1
Initial	86.52%	81.67%	82.77%
1st Incremental	80.85%	85.15%	81.53%
2nd Incremental	81.56%	85.98%	82.45%
From Scratch	62.03%	75.15%	65.30%

Separating the Metastealer family, with 80,000 examples available, to be used as a new class, the initial average accuracy obtained by the model was 86.53% and at the end of the second stage of incremental training, with the new class added to the model, it was 81.27%; therefore, there was a loss of accuracy of 5.27%, but 22.07% higher compared to the accuracy of the model trained without transfer learning with the reduced set of examples. The recall in the initial stage of the model with 60 families was 80.54% and rose to 84.33% after the second stage of incremental training, a gain in this metric of 3.79%, and a gain of

10.38% compared to the trained model with no transfer of learning. The F1 score in the initial stage was 81.76% and became 81.27% at the end of the second stage of incremental class training, which is a loss of 0.49%, but a gain about the model trained from scratch without the incremental process of 18.76%.

When it was separated from the Suppobox family, with 10,350 examples in the dataset, as an incremental class, the initial average accuracy obtained by the model was 86.72% and at the end of the second step of incremental training, with the new class added to the model, it was 82.00%, therefore showing a loss of accuracy of 4.72%, but 24.84% higher compared to the accuracy of the model trained without transfer learning with the reduced set of examples. The recall in the initial stage of the model with 60 families was 82.53% and rose to 84.88% after the second stage of incremental training, showing a gain in this metric of 2.35% and a gain of 11.65% compared to the trained model without transfer learning. The F1 score in the initial stage was 82.79% and became 81.51% at the end of the second stage of incremental class training, showing a loss of 1.28%, but a gain in the model trained from scratch without the incremental process of 21.13%.

For tests using the Darkshell family, with 1049 examples in the dataset, as an incremental class, the initial average accuracy obtained by the model was 86.52% and at the end of the second step of incremental training, with the new class added to the model, was 81.56%, therefore showing a loss of accuracy of 4.96%, but 19.52% higher compared to the accuracy of the model trained without transfer learning with the reduced set of examples. The recall in the initial stage of the model with 60 families was 81.67% and rose to 85.88% after the second stage of incremental training. There was a gain in this metric of 4.31%, and a gain of 10.84% when compared to the trained model without transfer of learning. The F1 score in the initial stage was 82.77% and became 82.45% at the end of the second stage of incremental class training, showing a loss of 0.31%, but there was a gain in the model trained from scratch, without the incremental process of 17.15%.

4. Conclusions

This work presented a computational scheme for DGA classification, composed of a deep learning model based on deep convolutional neural network and natural language processing and a technique for class increment by transfer learning to allow the increment of a new DGA family to the multiclass classifier model, avoiding catastrophic forgetting and maintaining the levels of the class metrics already known by the model.

First, observing the results presented in Section 3, it is possible to see that the proposed Model 1 obtained metrics very well aligned with the state-of-the-art research related to the multiclass classification of DGA, including obtaining metrics superior to some models obtained from the literature in direct comparison, using the same dataset for training and testing. These results reinforce the robustness of the work presented and its positioning in recent literature for the classification of DGA. The results obtained by the proposed Model 1 were also superior, on average, by around 19.5% to the results obtained by Model 2, a modified model, without the embedding layer in the data input, thus demonstrating the impact of applying character-level embedding on the results of the models presented.

Regarding the class increment technique, it was noticed that, in general, the number of examples available for the new class that should be added to the model does not have a significant impact on the general metrics after applying the class increment technique by transfer learning, since the results obtained by the three families tested as an incremental class with different numbers of examples in the dataset were very close.

It is also possible to verify that the proposed class increment technique proved to be efficient, keeping the levels of the model's metrics very close to those obtained at the time of initial training, and also much higher than the metrics obtained by the model trained exclusively with the reduced dataset, which reinforces the thesis that there was transfer of learning during the class increase process. Likewise, it can be concluded that the presented technique proved to be efficient in avoiding catastrophic forgetting in the incremental training of the proposed model for classifying examples from 61 DGA families.

Thus, the proposed computational scheme, combining the proposed deep learning model with the presented class increment technique, proved to be efficient both in classifying DGA families and robust and reliable in incrementing a new family to the multiclass classifier, avoiding catastrophic forgetting satisfactorily and keeping metrics balanced and at high levels.

For future work, the application of deep neural network techniques and models to seek better results in DGA multiclass classification can be suggested, since the general metrics are not yet at levels close to the limit, performing at around 80%. For example, the application of Large Language Models (LLMs), which have presented outstanding results in the area of Natural Language Processing (NLP). Possibly, Graph Neural Networks (GNNs) could be an innovative alternative to searching for relationships (edges) between domain names (vertices) of the same family. It would also be interesting to apply unsupervised learning techniques in order to find the occurrence of new DGA families in an automated way. This work presented results obtained with the increment of a class to a previously trained model. It may be interesting to evaluate the degradation of the metrics with the increment of more than one class. The integration of classifier models in real-world environments, through Passive DNS, would also be an interesting initiative, in order to observe and validate the performance of artificial intelligence models in a real environment.

Author Contributions: Conceptualization, J.R.G. and A.M.C.; methodology, J.R.G.; software, J.R.G.; validation, J.R.G., A.M.C. and L.A.N.; formal analysis, J.R.G., A.M.C. and L.A.N.; investigation, J.R.G. and A.M.C.; resources, J.R.G., A.M.C. and L.A.N.; data curation, J.R.G.; writing—original draft preparation, J.R.G.; writing—review and editing, J.R.G., A.M.C. and L.A.N.; visualization, J.R.G., A.M.C. and L.A.N.; supervision, A.M.C. and L.A.N.; project administration, A.M.C.; funding acquisition, A.M.C. and L.A.N. All authors have read and agreed to the published version of the manuscript.

Funding: This research was financed in part by the National Council for Scientific and Technological Development CNPq (Grant #313643/2021-0), and NIC.BR - Núcleo de Informação e Coordenação do Ponto BR (Grant FUNDUNESP #3467/2023).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The codes with the DGA multiclass classification experiment, the proposed models, and the codes of the compared models, as well as the dataset used, are published and available at https://www.kaggle.com/code/rafaelgregrio/multiclass-dga-classification-models-comparison (accessed on 16 August 2024). The codes referring to the class increment process by transfer learning, the results obtained, and the dataset used are available at https://www.kaggle.com/code/rafaelgregrio/multiclass-classifier-incremental-learning-dga (accessed on 16 August 2024).

Acknowledgments: We greatly thank the Fraunhofer Institute for Communication, Information Processing, and Ergonomics (FKIE) for providing access to the DGA feeds used in this work.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

BiLSTM	Bidirational Long Short-Term Memory
CNN	Convolutional Neural Network
C2	Command and Control
DGA	Domain Generation Algorithm
GNN	Graph Neural Network
LLM	Large Language Models
NLP	Natural Language Processing

Appendix A

Table A1. Model 1 on classic training method precision, recall, and F1 score of each of the 61 DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	99.86%	100.00%	99.93%	orchard	99.56%	99.27%	99.41%
banjori	99.56%	100.00%	99.78%	padcrypt	98.38%	97.05%	97.71%
bazarloader	99.96%	100.00%	99.98%	phorpiex	94.00%	79.34%	86.02%
chinad	99.77%	99.41%	99.59%	pitou	98.86%	97.51%	98.18%
conficker	57.82%	28.75%	38.25%	proslikefan	85.57%	14.08%	23.91%
corebot	99.30%	99.34%	99.31%	pseudoman	77.67%	91.23%	83.88%
cryptolocker	55.66%	44.31%	49.32%	pushdo	94.77%	97.22%	95.93%
darkshell	99.89%	100.00%	99.94%	pushdotid	91.84%	98.59%	95.09%
darkwatchman	99.94%	100.00%	99.97%	pykspa2s	97.07%	98.65%	97.85%
dircrypt	0.00%	0.00%	0.00%	qadars	98.98%	98.89%	98.94%
dyre	99.96%	100.00%	99.98%	qakbot	71.23%	74.64%	72.89%
ebury	99.22%	99.11%	99.17%	qsnatch	98.28%	98.62%	98.45%
emotet	92.88%	99.65%	96.14%	ramdo	99.39%	99.98%	99.68%
flubot	90.51%	95.16%	92.77%	ramnit	55.25%	60.23%	57.62%
fobber	43.74%	15.40%	22.49%	ranbyus	81.68%	94.05%	87.40%
gameover	99.97%	99.99%	99.98%	rovnix	99.38%	97.72%	98.54%
gameover_p2p	95.55%	54.28%	69.22%	shifu	80.09%	94.84%	86.78%
gozi	88.01%	68.44%	76.99%	simda	99.18%	99.80%	99.49%
grandoreiro	99.58%	99.89%	99.74%	sphinx	39.98%	41.19%	40.32%
locky	78.37%	39.95%	52.87%	suppobox	97.72%	99.68%	98.69%
m0yvtdd	54.61%	67.68%	59.34%	symmi	99.83%	99.93%	99.88%
metastealer	99.94%	100.00%	99.97%	tinba	94.21%	99.29%	96.69%
monerominer	99.95%	100.00%	99.97%	tinyfluff	99.93%	99.90%	99.91%
murofet	78.90%	94.90%	86.15%	tinynuke	99.44%	100.00%	99.72%
murofetweekly	99.85%	99.56%	99.70%	tufik	0.00%	0.00%	0.00%
mydoom	90.36%	92.24%	91.21%	urlzone	97.40%	92.34%	94.80%
necro	99.47%	98.89%	99.18%	vawtrak	87.03%	75.70%	80.90%
necurs	94.21%	88.53%	91.28%	virut	99.65%	99.99%	99.82%
ngioweb	96.19%	93.26%	94.68%	xxhex	99.90%	99.98%	99.94%
nymaim	54.71%	19.67%	28.90%	zloader	94.75%	99.80%	97.21%
nymaim2	93.19%	68.81%	79.15%	Average	86.75%	83.06%	83.78%

Table A2. Model 2 on classic training method precision, recall, and F1 score of each of the 61 DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	99.71%	99.93%	99.82%	orchard	99.02%	99.11%	99.06%
banjori	98.55%	99.96%	99.25%	padcrypt	87.76%	82.19%	84.01%
bazarloader	99.99%	100.00%	100.00%	phorpiex	48.84%	3.04%	5.67%
chinad	99.68%	99.37%	99.52%	pitou	84.30%	93.22%	88.26%
conficker	63.38%	19.32%	29.56%	proslikefan	74.66%	8.29%	14.74%
corebot	95.16%	90.31%	92.65%	pseudoman	59.74%	95.06%	73.34%
cryptolocker	64.92%	27.52%	38.57%	pushdo	88.20%	68.96%	77.38%
darkshell	99.90%	100.00%	99.95%	pushdotid	47.97%	65.56%	55.29%
darkwatchman	99.95%	100.00%	99.98%	pykspa2s	82.88%	86.63%	84.60%
dircrypt	0.00%	0.00%	0.00%	qadars	78.75%	65.90%	71.18%
dyre	99.97%	100.00%	99.99%	qakbot	70.71%	63.89%	67.09%
ebury	98.10%	99.66%	98.87%	qsnatch	98.35%	98.42%	98.39%
emotet	91.66%	99.58%	95.45%	ramdo	69.18%	77.16%	72.87%
flubot	88.38%	94.02%	91.10%	ramnit	42.62%	53.00%	47.15%
fobber	7.21%	0.43%	0.82%	ranbyus	77.06%	92.58%	84.10%
gameover	99.82%	99.89%	99.86%	rovnix	98.55%	96.99%	97.76%
gameover_p2p	91.09%	52.46%	66.54%	shifu	79.48%	33.84%	47.47%

Table A2. *Cont.*

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
gozi	77.12%	8.81%	15.57%	simda	77.36%	97.57%	86.27%
grandoreiro	99.10%	99.16%	99.13%	sphinx	43.02%	13.71%	17.18%
locky	78.77%	34.10%	47.47%	suppobox	64.08%	67.10%	64.79%
m0yvtdd	44.48%	88.82%	59.13%	symmi	99.38%	100.00%	99.69%
metastealer	99.76%	100.00%	99.88%	tinba	88.48%	99.31%	93.58%
monerominer	99.97%	100.00%	99.98%	tinyfluff	99.65%	99.87%	99.76%
murofet	74.33%	94.47%	83.20%	tinynuke	99.73%	100.00%	99.87%
murofetweekly	99.71%	99.45%	99.57%	tufik	0.00%	0.00%	0.00%
mydoom	60.14%	41.17%	48.69%	urlzone	92.36%	88.43%	90.33%
necro	99.11%	94.77%	96.89%	vawtrak	64.96%	20.58%	30.57%
necurs	92.14%	84.74%	88.28%	virut	99.69%	100.00%	99.84%
ngioweb	95.92%	71.51%	81.93%	xxhex	99.90%	99.92%	99.91%
nymaim	46.82%	9.30%	15.38%	zloader	90.74%	99.89%	95.09%
nymaim2	94.33%	37.03%	53.11%	Average	79.78%	72.39%	72.87%

Table A3. Bilbo on classic training method precision, recall, and F1 score of each of the 61 DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	99.63%	62.49%	75.31%	orchard	92.61%	19.95%	32.15%
banjori	78.91%	94.73%	86.04%	padcrypt	88.59%	87.07%	87.78%
bazarloader	98.26%	99.32%	98.79%	phorpiex	0.00%	0.00%	0.00%
chinad	98.82%	99.38%	99.10%	pitou	75.70%	34.76%	45.16%
conficker	14.65%	0.58%	1.11%	proslikefan	0.00%	0.00%	0.00%
corebot	96.39%	85.58%	90.65%	pseudoman	34.53%	2.74%	5.07%
cryptolocker	55.01%	16.25%	24.97%	pushdo	52.81%	21.77%	28.03%
darkshell	89.44%	82.97%	86.07%	pushdotid	20.60%	53.77%	29.73%
darkwatchman	85.77%	100.00%	92.34%	pykspa2s	45.83%	8.28%	13.14%
dircrypt	0.00%	0.00%	0.00%	qadars	99.40%	77.19%	86.89%
dyre	99.95%	100.00%	99.98%	qakbot	56.46%	59.50%	57.94%
ebury	94.38%	75.73%	84.02%	qsnatch	90.65%	98.76%	94.53%
emotet	0.00%	0.00%	0.00%	ramdo	79.18%	80.91%	79.95%
flubot	78.51%	92.21%	84.77%	ramnit	16.29%	8.57%	11.23%
fobber	0.00%	0.00%	0.00%	ranbyus	61.49%	89.22%	72.81%
gameover	99.83%	100.00%	99.91%	rovnix	87.07%	91.01%	88.98%
gameover_p2p	95.75%	28.09%	43.31%	shifu	0.00%	0.00%	0.00%
gozi	68.27%	27.34%	38.61%	simda	55.89%	81.17%	66.20%
grandoreiro	99.79%	95.87%	97.79%	sphinx	31.29%	34.51%	32.56%
locky	57.67%	2.65%	5.02%	suppobox	65.30%	82.98%	73.00%
m0yvtdd	0.00%	0.00%	0.00%	symmi	95.10%	99.90%	97.44%
metastealer	99.98%	98.77%	99.37%	tinba	84.40%	99.51%	91.33%
monerominer	99.85%	100.00%	99.92%	tinyfluff	99.60%	99.82%	99.71%
murofet	71.48%	97.20%	82.36%	tinynuke	76.87%	100.00%	86.67%
murofetweekly	99.70%	98.93%	99.32%	tufik	0.00%	0.00%	0.00%
mydoom	0.00%	0.00%	0.00%	urlzone	93.93%	92.02%	92.97%
necro	100.00%	98.10%	99.04%	vawtrak	0.00%	0.00%	0.00%
necurs	72.80%	66.00%	69.22%	virut	99.48%	99.99%	99.73%
ngioweb	93.55%	71.35%	80.91%	xxhex	98.97%	100.00%	99.48%
nymaim	0.00%	0.00%	0.00%	zloader	79.64%	97.73%	87.73%
nymaim2	97.49%	36.81%	53.38%	Average	64.39%	58.22%	58.22%

Table A4. DistilBERT on classic training method precision, recall, and F1 score of each of the 61 DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	0.00%	0.00%	0.00%	orchard	87.46%	17.54%	28.78%
banjori	89.00%	73.64%	80.54%	padcrypt	79.43%	69.77%	73.32%
bazarloader	99.37%	98.10%	98.72%	phorpiex	85.97%	30.49%	44.94%
chinad	0.00%	0.00%	0.00%	pitou	79.66%	27.28%	38.90%
conficker	26.80%	5.41%	8.32%	proslikefan	95.24%	0.81%	1.58%
corebot	33.33%	0.02%	0.04%	pseudoman	60.55%	25.57%	35.92%
cryptolocker	0.00%	0.00%	0.00%	pushdo	73.68%	80.67%	76.94%
darkshell	93.32%	54.78%	68.88%	pushdotid	47.76%	14.39%	21.70%
darkwatchman	93.50%	97.32%	95.37%	pykspa2s	62.18%	28.82%	38.58%
dircrypt	0.00%	0.00%	0.00%	qadars	78.73%	0.72%	1.43%
dyre	82.71%	42.22%	55.62%	qakbot	42.74%	15.65%	22.79%
ebury	16.67%	0.06%	0.11%	qsnatch	92.26%	92.63%	92.44%
emotet	0.00%	0.00%	0.00%	ramdo	33.33%	0.02%	0.04%
flubot	24.86%	1.41%	2.35%	ramnit	29.15%	5.15%	8.70%
fobber	5.13%	0.10%	0.20%	ranbyus	0.00%	0.00%	0.00%
gameover	76.76%	96.06%	85.32%	rovnix	0.00%	0.00%	0.00%
gameover_p2p	0.00%	0.00%	0.00%	shifu	53.94%	44.41%	47.51%
gozi	89.12%	29.29%	43.71%	simda	70.53%	91.50%	79.64%
grandoreiro	88.61%	78.56%	83.28%	sphinx	0.00%	0.00%	0.00%
locky	47.76%	0.73%	1.43%	suppobox	93.27%	98.08%	95.61%
m0yvtdd	43.81%	54.23%	47.89%	symmi	90.74%	72.72%	80.49%
metastealer	72.33%	98.84%	83.47%	tinba	39.85%	74.88%	51.89%
monerominer	73.05%	97.49%	83.47%	tinyfluff	95.35%	98.83%	97.06%
murofet	40.29%	77.02%	52.65%	tinynuke	0.00%	0.00%	0.00%
murofetweekly	59.77%	6.50%	10.79%	tufik	0.00%	0.00%	0.00%
mydoom	87.77%	35.99%	49.61%	urlzone	66.29%	30.45%	40.41%
necro	33.33%	0.02%	0.05%	vawtrak	91.39%	4.54%	8.56%
necurs	35.76%	35.90%	35.72%	virut	90.91%	100.00%	95.24%
ngioweb	96.83%	79.41%	87.26%	xxhex	97.60%	99.93%	98.75%
nymaim	32.21%	1.98%	3.62%	zloader	0.00%	0.00%	0.00%
nymaim2	95.96%	45.89%	61.49%	Average	53.71%	36.65%	38.05%

Table A5. CNN on classic training method precision, recall, and F1 score of each of the 61 DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	98.95%	96.29%	97.60%	orchard	94.39%	62.04%	74.47%
banjori	96.41%	99.75%	98.05%	padcrypt	87.22%	78.24%	82.40%
bazarloader	100.00%	100.00%	100.00%	phorpiex	85.13%	31.99%	46.31%
chinad	81.41%	84.78%	83.03%	pitou	89.46%	80.30%	84.61%
conficker	41.25%	22.89%	29.39%	proslikefan	91.65%	7.81%	14.37%
corebot	94.92%	84.38%	89.23%	pseudoman	59.10%	30.52%	39.91%
cryptolocker	68.29%	18.66%	29.11%	pushdo	82.52%	90.42%	86.26%
darkshell	97.73%	98.01%	97.85%	pushdotid	76.74%	77.74%	77.17%
darkwatchman	99.57%	100.00%	99.79%	pykspa2s	81.52%	83.22%	82.35%
dircrypt	0.00%	0.00%	0.00%	qadars	94.25%	88.29%	91.15%
dyre	99.89%	100.00%	99.94%	qakbot	63.71%	63.25%	63.47%
ebury	94.24%	81.40%	87.19%	qsnatch	96.85%	98.02%	97.43%
emotet	61.90%	87.81%	72.47%	ramdo	97.76%	99.91%	98.82%
flubot	68.14%	81.04%	74.01%	ramnit	37.28%	24.96%	29.87%
fobber	0.00%	0.00%	0.00%	ranbyus	63.88%	58.33%	60.91%
gameover	98.49%	99.49%	98.99%	rovnix	44.44%	0.12%	0.23%
gameover_p2p	86.51%	15.51%	26.17%	shifu	56.35%	76.53%	64.76%
gozi	77.20%	37.79%	50.42%	simda	92.11%	98.86%	95.36%
grandoreiro	95.42%	89.93%	92.47%	sphinx	0.00%	0.00%	0.00%
locky	73.41%	33.14%	45.59%	suppobox	84.70%	95.31%	89.66%

Table A5. *Cont.*

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
m0yvtdd	45.86%	76.42%	57.28%	symmi	99.78%	100.00%	99.89%
metastealer	99.87%	100.00%	99.93%	tinba	76.55%	92.01%	83.57%
monerominer	99.53%	99.96%	99.75%	tinyfluff	98.23%	99.80%	99.01%
murofet	69.87%	89.85%	78.61%	tinynuke	99.62%	100.00%	99.81%
murofetweekly	96.34%	77.99%	85.13%	tufik	0.00%	0.00%	0.00%
mydoom	85.39%	36.06%	50.23%	urlzone	94.21%	91.48%	92.82%
necro	99.00%	93.39%	96.10%	vawtrak	48.56%	31.12%	37.90%
necurs	92.32%	79.20%	85.26%	virut	98.38%	99.98%	99.17%
ngioweb	92.40%	82.23%	87.02%	xxhex	99.50%	99.93%	99.71%
nymaim	53.02%	13.62%	21.64%	zloader	47.61%	77.75%	59.03%
nymaim2	91.62%	53.80%	67.60%	Average	77.06%	68.38%	69.68%

Table A6. CNN+LSTM on classic training method precision, recall, and F1 score of each of the 61 DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	99.29%	89.25%	93.81%	orchard	93.47%	94.41%	93.84%
banjori	95.05%	99.97%	97.45%	padcrypt	73.10%	82.20%	76.39%
bazarloader	99.99%	99.95%	99.97%	phorpiex	18.18%	0.48%	0.94%
chinad	98.99%	98.81%	98.89%	pitou	87.62%	51.62%	63.14%
conficker	28.38%	16.63%	20.76%	proslikefan	3.70%	0.06%	0.11%
corebot	95.37%	93.57%	94.44%	pseudoman	34.73%	72.96%	45.78%
cryptolocker	48.64%	37.84%	42.26%	pushdo	76.40%	83.82%	79.76%
darkshell	91.02%	82.52%	86.53%	pushdotid	48.33%	66.89%	55.98%
darkwatchman	99.39%	99.32%	99.35%	pykspa2s	67.89%	73.99%	70.35%
dircrypt	0.00%	0.00%	0.00%	qadars	90.96%	80.37%	85.19%
dyre	100.00%	100.00%	100.00%	qakbot	70.31%	68.79%	69.50%
ebury	96.02%	95.53%	95.76%	qsnatch	96.98%	97.47%	97.21%
emotet	80.78%	97.59%	88.31%	ramdo	93.79%	99.59%	96.57%
flubot	87.28%	95.22%	91.05%	ramnit	46.68%	56.95%	51.14%
fobber	12.12%	0.21%	0.42%	ranbyus	77.53%	91.03%	83.74%
gameover	99.92%	99.94%	99.93%	rovnix	97.52%	95.96%	96.72%
gameover_p2p	92.45%	43.45%	58.92%	shifu	63.18%	60.61%	60.68%
gozi	77.80%	33.69%	45.86%	simda	84.84%	97.62%	90.77%
grandoreiro	96.68%	98.14%	97.39%	sphinx	34.96%	30.48%	30.74%
locky	35.11%	3.77%	6.79%	suppobox	73.71%	87.09%	79.82%
m0yvtdd	50.82%	29.90%	37.29%	symmi	98.04%	99.83%	98.92%
metastealer	99.88%	99.99%	99.93%	tinba	92.46%	99.26%	95.74%
monerominer	99.85%	99.94%	99.90%	tinyfluff	98.82%	99.57%	99.19%
murofet	78.21%	92.46%	84.68%	tinynuke	92.25%	99.51%	95.65%
murofetweekly	99.96%	99.70%	99.83%	tufik	0.00%	0.00%	0.00%
mydoom	0.00%	0.00%	0.00%	urlzone	95.39%	90.20%	92.71%
necro	99.78%	96.62%	98.18%	vawtrak	22.94%	4.66%	7.40%
necurs	89.64%	86.30%	87.94%	virut	99.66%	100.00%	99.83%
ngioweb	96.56%	67.51%	79.46%	xxhex	99.47%	99.73%	99.60%
nymaim	32.12%	1.62%	3.03%	zloader	94.24%	99.59%	96.84%
nymaim2	95.15%	42.99%	59.22%	Average	73.83%	70.28%	70.19%

Table A7. RNN on classic training method precision, recall, and F1 score of each of the 61 DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	56.79%	30.52%	37.44%	orchard	54.47%	14.80%	23.19%
banjori	49.10%	51.23%	49.98%	padcrypt	47.09%	36.13%	39.60%
bazarloader	90.66%	98.45%	94.07%	phorpiex	0.00%	0.00%	0.00%

Table A7. *Cont.*

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
chinad	71.62%	69.84%	70.44%	pitou	54.09%	24.75%	30.21%
conficker	12.19%	5.52%	6.60%	proslikefan	0.00%	0.00%	0.00%
corebot	69.34%	44.36%	47.49%	pseudoman	11.27%	7.22%	8.80%
cryptolocker	24.08%	11.44%	15.47%	pushdo	27.26%	20.29%	23.27%
darkshell	32.30%	17.44%	22.66%	pushdotid	20.34%	21.05%	18.41%
darkwatchman	78.20%	93.95%	85.35%	pykspa2s	15.14%	2.64%	4.50%
dircrypt	0.00%	0.00%	0.00%	qadars	51.40%	48.05%	49.62%
dyre	91.88%	99.93%	95.40%	qakbot	47.51%	46.58%	46.40%
ebury	57.40%	40.75%	47.52%	qsnatch	92.52%	90.89%	91.34%
emotet	49.51%	59.78%	54.12%	ramdo	60.67%	62.16%	61.38%
flubot	76.25%	86.30%	80.67%	ramnit	14.08%	14.13%	14.10%
fobber	0.00%	0.00%	0.00%	ranbyus	65.71%	76.18%	69.83%
gameover	99.09%	99.59%	99.34%	rovnix	65.38%	62.70%	64.00%
gameover_p2p	29.71%	5.78%	9.68%	shifu	22.39%	15.64%	16.71%
gozi	22.27%	8.66%	12.47%	simda	66.55%	75.76%	70.81%
grandoreiro	54.92%	50.29%	52.25%	sphinx	19.54%	4.59%	7.07%
locky	0.00%	0.00%	0.00%	suppobox	21.87%	23.63%	22.72%
m0yvtdd	2.08%	0.13%	0.24%	symmi	58.55%	57.59%	58.06%
metastealer	97.49%	99.99%	98.69%	tinba	80.52%	98.99%	88.57%
moneromine r	86.79%	94.41%	90.10%	tinyfluff	93.89%	99.59%	96.52%
murofet	63.39%	90.67%	74.55%	tinynuke	80.71%	62.32%	53.67%
murofetweekly	89.09%	89.88%	88.01%	tufik	0.00%	0.00%	0.00%
mydoom	0.00%	0.00%	0.00%	urlzone	75.66%	84.37%	79.44%
necro	65.09%	59.39%	62.06%	vawtrak	0.00%	0.00%	0.00%
necurs	57.35%	67.88%	61.80%	virut	97.99%	100.00%	98.97%
ngioweb	58.53%	22.95%	28.53%	xxhex	66.07%	65.84%	65.96%
nymaim	7.41%	0.04%	0.08%	zloader	57.51%	63.66%	60.36%
nymaim2	26.48%	11.78%	16.31%	Average	47.33%	44.11%	43.69%

Table A8. Class Incremental Method, initial 60 DGA families without Metastealer precision, recall, and F1 score of each DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	100.00%	99.79%	99.89%	padcrypt	99.06%	99.20%	99.13%
banjori	99.02%	99.97%	99.49%	phorpiex	90.46%	57.14%	70.04%
bazarloader	99.90%	100.00%	99.95%	pitou	97.73%	97.51%	97.62%
chinad	99.91%	99.51%	99.71%	proslikefan	100.00%	11.25%	20.22%
conficker	56.82%	27.01%	36.61%	pseudoman	71.72%	60.25%	65.49%
corebot	99.42%	99.71%	99.57%	pushdo	94.29%	98.04%	96.13%
cryptolocker	67.61%	32.60%	43.99%	pushdotid	92.39%	97.63%	94.94%
darkshell	98.75%	73.37%	84.19%	pykspa2s	95.28%	99.33%	97.26%
darkwatchman	100.00%	100.00%	100.00%	qadars	99.21%	98.06%	98.63%
dircrypt	0.00%	0.00%	0.00%	qakbot	68.76%	78.96%	73.51%
dyre	99.95%	100.00%	99.98%	qsnatch	97.99%	98.56%	98.28%
ebury	99.83%	96.81%	98.29%	ramdo	98.23%	99.67%	98.95%
emotet	89.12%	100.00%	94.25%	ramnit	53.22%	59.99%	56.40%
flubot	91.46%	94.46%	92.94%	ranbyus	83.61%	92.04%	87.62%
fobber	40.59%	6.80%	11.65%	rovnix	100.00%	98.76%	99.38%
gameover	99.99%	99.98%	99.98%	shifu	80.45%	95.79%	87.45%
gameover_p2p	95.94%	54.43%	69.46%	simda	97.51%	99.90%	98.69%
gozi	89.74%	58.17%	70.59%	sphinx	40.79%	49.45%	44.70%
grandoreiro	99.68%	100.00%	99.84%	suppobox	95.39%	99.48%	97.39%
locky	83.50%	40.00%	54.09%	symmi	99.76%	99.92%	99.84%
m0yvtdd	54.22%	58.14%	56.11%	tinba	94.49%	99.22%	96.80%
moneromine r	99.95%	100.00%	99.97%	tinyfluff	99.78%	99.88%	99.83%
murofet	77.89%	95.89%	85.96%	tinynuke	99.84%	100.00%	99.92%

Table A8. *Cont.*

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
murofetweekly	100.00%	99.89%	99.95%	tufik	0.00%	0.00%	0.00%
mydoom	95.11%	72.91%	82.54%	urlzone	98.16%	92.71%	95.35%
necro	99.93%	99.10%	99.51%	vawtrak	78.29%	82.83%	80.49%
necurs	94.59%	89.12%	91.77%	virut	99.41%	99.98%	99.70%
ngioweb	94.39%	90.86%	92.59%	xxhex	99.92%	100.00%	99.96%
nymaim	64.36%	10.63%	18.25%	zloader	94.50%	99.96%	97.15%
nymaim2	81.87%	68.19%	74.41%				
orchard	98.20%	99.77%	98.98%	Average	86.53%	80.54%	81.76%

Table A9. Class Incremental Method, 1st incremental train with Metastealer examples precision, recall, and F1 score of each DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	100.00%	99.79%	99.89%	padcrypt	98.28%	99.60%	98.94%
banjori	99.63%	99.32%	99.48%	phorpiex	77.12%	66.96%	71.68%
bazarloader	99.91%	100.00%	99.95%	pitou	90.76%	97.96%	94.22%
chinad	99.91%	99.35%	99.63%	proslikefan	26.30%	20.93%	23.31%
conficker	47.80%	29.60%	36.56%	pseudoman	46.76%	96.89%	63.08%
corebot	95.88%	99.48%	97.65%	pushdo	96.09%	97.00%	96.54%
cryptolocker	48.13%	53.81%	50.81%	pushdotid	89.76%	97.08%	93.28%
darkshell	94.42%	73.37%	82.58%	pykspa2s	96.00%	96.64%	96.32%
darkwatchman	99.91%	100.00%	99.96%	qadars	97.96%	98.67%	98.31%
dircrypt	5.32%	34.57%	9.22%	qakbot	76.71%	65.74%	70.80%
dyre	99.98%	100.00%	99.99%	qsnatch	99.39%	97.65%	98.51%
ebury	95.02%	99.33%	97.12%	ramdo	96.17%	99.73%	97.92%
emotet	69.52%	100.00%	82.02%	ramnit	57.51%	46.95%	51.70%
flubot	90.26%	94.93%	92.53%	ranbyus	80.71%	93.61%	86.68%
fobber	30.15%	26.20%	28.04%	rovnix	98.08%	99.29%	98.68%
gameover	99.99%	99.76%	99.88%	shifu	70.39%	98.04%	81.95%
gameover_p2p	75.86%	61.10%	67.69%	simda	98.00%	99.77%	98.88%
gozi	76.54%	63.88%	69.64%	sphinx	18.86%	95.05%	31.47%
grandoreiro	99.04%	100.00%	99.52%	suppobox	95.15%	98.85%	96.97%
locky	58.90%	35.97%	44.66%	symmi	99.06%	100.00%	99.53%
m0yvtdd	42.14%	85.14%	56.37%	tinba	94.58%	97.29%	95.91%
monerominer	99.93%	100.00%	99.96%	tinyfluff	99.69%	99.89%	99.79%
murofet	79.92%	43.67%	56.48%	tinynuke	99.84%	100.00%	99.92%
murofetweekly	100.00%	99.67%	99.84%	tufik	10.72%	47.22%	17.47%
mydoom	51.83%	85.59%	64.57%	urlzone	99.63%	92.14%	95.74%
necro	99.79%	97.64%	98.70%	vawtrak	71.34%	83.07%	76.76%
necurs	98.87%	83.47%	90.52%	virut	99.61%	99.98%	99.79%
ngioweb	93.33%	90.97%	92.14%	xxhex	99.63%	100.00%	99.81%
nymaim	32.67%	23.56%	27.38%	zloader	91.09%	99.77%	95.23%
nymaim2	78.20%	71.40%	74.64%	metastealer	97.16%	100.00%	98.56%
orchard	97.10%	99.77%	98.42%	Average	80.86%	84.21%	81.04%

Table A10. Class Incremental Method, 2nd incremental train with Metastealer examples precision, recall, and F1 score of each DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	100.00%	100.00%	100.00%	padcrypt	97.51%	99.60%	98.54%
banjori	99.45%	99.96%	99.70%	phorpiex	90.31%	83.26%	86.64%
bazarloader	99.87%	100.00%	99.94%	pitou	93.33%	98.41%	95.81%
chinad	99.71%	99.09%	99.40%	proslikefan	54.11%	19.38%	28.54%
conficker	50.10%	32.20%	39.20%	pseudoman	56.19%	95.05%	70.63%

Table A10. *Cont.*

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
corebot	95.58%	99.77%	97.63%	pushdo	93.84%	94.79%	94.31%
cryptolocker	44.76%	48.65%	46.62%	pushdotid	89.19%	97.69%	93.24%
darkshell	97.93%	73.37%	83.89%	pykspa2s	96.38%	96.41%	96.39%
darkwatchman	99.30%	100.00%	99.65%	qadars	95.94%	98.44%	97.18%
dircrypt	4.41%	36.19%	7.87%	qakbot	74.46%	64.99%	69.40%
dyre	100.00%	100.00%	100.00%	qsnatch	98.77%	97.47%	98.11%
ebury	92.96%	99.83%	96.27%	ramdo	96.99%	100.00%	98.47%
emotet	71.26%	99.77%	83.14%	ramnit	50.98%	47.51%	49.18%
flubot	89.91%	92.35%	91.11%	ranbyus	78.74%	90.58%	84.24%
fobber	29.55%	21.56%	24.93%	rovnix	91.67%	99.12%	95.25%
gameover	100.00%	99.78%	99.89%	shifu	65.92%	98.74%	79.06%
gameover_p2p	74.24%	59.38%	65.98%	simda	98.59%	99.18%	98.88%
gozi	77.56%	67.68%	72.28%	sphinx	21.84%	87.73%	34.98%
grandoreiro	99.04%	100.00%	99.52%	suppobox	96.05%	98.56%	97.29%
locky	55.48%	38.35%	45.36%	symmi	97.00%	99.68%	98.32%
m0yvtdd	43.19%	86.82%	57.68%	tinba	94.21%	92.78%	93.49%
moneromineR	99.56%	100.00%	99.78%	tinyfluff	99.29%	100.00%	99.64%
murofet	80.28%	39.84%	53.25%	tinynuke	99.84%	100.00%	99.92%
murofetweekly	99.78%	100.00%	99.89%	tufik	10.49%	51.67%	17.45%
mydoom	70.71%	89.05%	78.83%	urlzone	99.70%	91.68%	95.52%
necro	99.65%	98.89%	99.27%	vawtrak	58.45%	83.68%	68.82%
necurs	96.84%	83.99%	89.96%	virut	99.70%	99.87%	99.79%
ngioweb	89.25%	91.41%	90.32%	xxhex	99.85%	100.00%	99.92%
nymaim	38.50%	24.41%	29.88%	zloader	90.03%	98.99%	94.29%
nymaim2	73.68%	76.89%	75.25%	metastealer	98.70%	100.00%	99.34%
orchard	96.67%	99.77%	98.20%	Average	81.27%	84.33%	81.27%

Table A11. Model 1, not pre-trained, trained with the same number of few examples used in the incremental method for Metastealer class, precision, recall, and F1 score of each DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	98.74%	100.00%	99.37%	padcrypt	59.22%	91.16%	71.80%
banjori	92.67%	96.46%	94.53%	phorpiex	40.18%	89.06%	55.38%
bazarloader	99.68%	99.46%	99.57%	pitou	50.94%	98.41%	67.13%
chinad	79.26%	93.17%	85.65%	proslikefan	4.78%	34.60%	8.40%
conficker	14.81%	28.18%	19.41%	pseudoman	19.29%	70.44%	30.28%
corebot	92.41%	92.78%	92.60%	pushdo	74.06%	71.93%	72.98%
cryptolocker	13.26%	23.24%	16.88%	pushdotid	40.08%	63.31%	49.08%
darkshell	88.49%	100.00%	93.90%	pykspa2s	91.56%	84.07%	87.65%
darkwatchman	98.36%	100.00%	99.17%	qadars	58.89%	89.06%	70.90%
dircrypt	1.14%	21.11%	2.17%	qakbot	35.32%	12.29%	18.24%
dyre	99.82%	100.00%	99.91%	qsnatch	96.44%	89.77%	92.98%
ebury	66.11%	99.33%	79.38%	ramdo	96.99%	100.00%	98.47%
emotet	25.59%	94.53%	40.28%	ramnit	12.66%	17.32%	14.63%
flubot	57.15%	58.62%	57.88%	ranbyus	45.23%	47.75%	46.46%
fobber	6.62%	43.12%	11.48%	rovnix	58.39%	94.70%	72.24%
gameover	99.64%	99.38%	99.51%	shifu	37.94%	75.04%	50.40%
gameover_p2p	16.06%	67.43%	25.94%	simda	88.08%	83.40%	85.68%
gozi	16.05%	68.82%	26.03%	sphinx	6.07%	55.49%	10.95%
grandoreiro	69.28%	100.00%	81.85%	suppobox	68.61%	86.03%	76.34%
locky	19.04%	15.03%	16.80%	symmi	82.34%	95.25%	88.33%
m0yvtdd	43.85%	75.06%	55.36%	tinba	80.24%	45.70%	58.23%
moneromineR	99.45%	99.97%	99.71%	tinyfluff	98.09%	99.35%	98.72%
murofet	60.62%	21.50%	31.74%	tinynuke	99.68%	100.00%	99.84%
murofetweekly	96.23%	98.13%	97.17%	tufik	8.55%	38.72%	14.01%
mydoom	40.98%	79.25%	54.03%	urlzone	70.21%	68.88%	69.54%

Table A11. *Cont.*

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
necro	37.72%	94.51%	53.92%	vawtrak	30.07%	65.78%	41.27%
necurs	85.82%	36.71%	51.42%	virut	98.73%	95.42%	97.05%
ngioweb	63.86%	84.47%	72.74%	xxhex	96.79%	99.70%	98.22%
nymaim	5.10%	15.72%	7.70%	zloader	55.74%	80.92%	66.01%
nymaim2	59.65%	62.24%	60.92%	metastealer	95.78%	100.00%	97.84%
orchard	62.32%	99.54%	76.65%	Average	59.19%	73.96%	62.50%

Table A12. Class Incremental Method, initial 60 DGA families without Suppobox precision, recall, and F1 score of each DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	99.79%	100.00%	99.89%	orchard	99.51%	99.75%	99.63%
banjori	99.96%	99.99%	99.97%	padcrypt	99.07%	99.21%	99.14%
bazarloader	99.89%	100.00%	99.95%	phorpiex	91.62%	72.57%	80.99%
chinad	99.59%	99.30%	99.45%	pitou	99.33%	97.37%	98.34%
conficker	62.28%	26.05%	36.74%	proslikefan	73.68%	2.34%	4.53%
corebot	99.06%	99.61%	99.33%	pseudoman	67.80%	94.66%	79.01%
cryptolocker	58.42%	39.49%	47.13%	pushdo	96.33%	97.40%	96.86%
darkshell	100.00%	100.00%	100.00%	pushdotid	93.89%	98.90%	96.33%
darkwatchman	99.78%	100.00%	99.89%	pykspa2s	98.71%	97.67%	98.19%
dircrypt	0.00%	0.00%	0.00%	qadars	98.78%	98.13%	98.45%
dyre	99.98%	100.00%	99.99%	qakbot	68.76%	77.97%	73.07%
ebury	99.11%	97.89%	98.50%	qsnatch	97.70%	98.73%	98.21%
emotet	91.39%	99.78%	95.40%	ramdo	99.50%	100.00%	99.75%
flubot	90.77%	95.27%	92.97%	ramnit	58.26%	54.27%	56.19%
fobber	46.15%	1.98%	3.79%	ranbyus	79.98%	94.30%	86.55%
gameover	99.97%	99.99%	99.98%	rovnix	98.91%	98.55%	98.73%
gameover_p2p	95.02%	53.36%	68.34%	shifu	79.46%	97.54%	87.57%
gozi	83.51%	74.25%	78.61%	simda	99.08%	99.92%	99.50%
grandoreiro	100.00%	100.00%	100.00%	sphinx	40.30%	55.86%	46.82%
locky	74.23%	42.62%	54.15%	symmi	99.92%	100.00%	99.96%
m0yvtdd	57.98%	63.01%	60.39%	tinba	94.30%	99.07%	96.63%
metastealer	99.96%	100.00%	99.98%	tinyfluff	99.93%	99.94%	99.94%
monerominer	99.99%	99.99%	99.99%	tinynuke	100.00%	100.00%	100.00%
murofet	80.51%	94.46%	85.97%	tufik	33.33%	0.02%	0.04%
murofetweekly	99.89%	99.68%	99.79%	urlzone	97.14%	92.43%	94.72%
mydoom	92.31%	85.25%	88.64%	vawtrak	86.00%	84.51%	85.25%
necro	99.86%	99.44%	99.65%	virut	99.60%	99.99%	99.79%
necurs	94.47%	88.69%	91.49%	xxhex	99.26%	100.00%	99.63%
ngioweb	94.67%	94.88%	94.77%	zloader	93.85%	99.52%	96.60%
nymaim	52.04%	19.94%	28.83%				
nymaim2	82.59%	66.40%	73.61%	Average	86.62%	82.53%	82.79%

Table A13. Class Incremental Method, 1st incremental train with Suppobox examples precision, recall, and F1 score of each DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	98.75%	100.00%	99.37%	orchard	96.20%	99.51%	97.83%
banjori	99.35%	99.76%	99.55%	padcrypt	98.94%	98.81%	98.87%
bazarloader	99.91%	100.00%	99.96%	phorpiex	77.83%	68.36%	72.79%
chinad	97.66%	93.97%	95.78%	pitou	44.58%	96.49%	60.98%
conficker	43.21%	25.79%	32.30%	proslikefan	0.00%	0.00%	0.00%
corebot	98.82%	88.50%	93.37%	pseudoman	41.77%	97.92%	58.56%
cryptolocker	42.81%	32.60%	37.01%	pushdo	91.77%	95.77%	93.73%
darkshell	97.30%	100.00%	98.63%	pushdotid	94.43%	89.60%	91.95%

Table A13. *Cont.*

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
darkwatchman	99.96%	100.00%	99.98%	pykspa2s	96.98%	94.30%	95.63%
dircrypt	6.68%	27.29%	10.73%	qadars	97.27%	97.96%	97.61%
dyre	99.80%	100.00%	99.90%	qakbot	83.68%	44.11%	57.77%
ebury	98.09%	99.30%	98.69%	qsnatch	99.22%	96.01%	97.59%
emotet	82.78%	100.00%	90.58%	ramdo	93.22%	99.94%	96.46%
flubot	90.21%	94.85%	92.47%	ramnit	56.75%	44.66%	49.98%
fobber	44.64%	8.24%	13.91%	ranbyus	79.85%	83.53%	81.65%
gameover	100.00%	99.58%	99.79%	rovnix	71.91%	96.92%	82.56%
gameover_p2p	60.21%	58.05%	59.11%	shifu	48.23%	98.70%	64.80%
gozi	72.60%	9.96%	17.52%	simda	98.61%	91.64%	95.00%
grandoreiro	76.00%	100.00%	86.36%	sphinx	27.34%	89.14%	41.85%
locky	81.58%	22.99%	35.87%	symmi	98.79%	100.00%	99.39%
m0yvtdd	50.86%	60.72%	55.35%	tinba	95.19%	95.32%	95.25%
metastealer	99.95%	100.00%	99.97%	tinyfluff	99.77%	99.99%	99.88%
monerominer	99.96%	99.99%	99.98%	tinynuke	99.67%	100.00%	99.84%
murofet	80.70%	88.71%	84.52%	tufik	0.00%	0.00%	0.00%
murofetweekly	99.46%	98.93%	99.19%	urlzone	98.76%	90.37%	94.38%
mydoom	90.73%	77.60%	83.65%	vawtrak	78.50%	68.77%	73.32%
necro	98.95%	99.30%	99.13%	virut	99.37%	99.96%	99.67%
necurs	99.24%	73.64%	84.54%	xxhex	99.33%	99.85%	99.59%
ngioweb	92.83%	67.80%	78.37%	zloader	87.43%	98.59%	92.67%
nymaim	44.14%	16.81%	24.35%	suppobox	12.48%	96.94%	22.11%
nymaim2	57.17%	55.80%	56.48%	Average	78.23%	79.24%	76.03%

Table A14. Class Incremental Method, 2nd incremental train with Suppobox examples precision, recall, and F1 score of each DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	96.74%	100.00%	98.34%	orchard	99.27%	99.75%	99.51%
banjori	98.36%	99.92%	99.13%	padcrypt	97.15%	99.34%	98.23%
bazarloader	99.91%	100.00%	99.96%	phorpiex	76.77%	84.07%	80.25%
chinad	98.88%	98.99%	98.93%	pitou	95.47%	97.15%	96.30%
conficker	55.17%	29.39%	38.35%	proslikefan	100.00%	4.17%	8.01%
corebot	99.32%	97.49%	98.39%	pseudoman	56.87%	96.44%	71.55%
cryptolocker	40.65%	54.48%	46.56%	pushdo	91.25%	96.72%	93.91%
darkshell	98.48%	100.00%	99.23%	pushdotid	94.62%	95.82%	95.21%
darkwatchman	100.00%	100.00%	100.00%	pykspa2s	94.05%	96.36%	95.19%
dircrypt	4.48%	42.82%	8.11%	qadars	96.45%	98.68%	97.55%
dyre	99.93%	100.00%	99.97%	qakbot	72.39%	65.83%	68.95%
ebury	91.91%	99.82%	95.70%	qsnatch	98.72%	97.74%	98.22%
emotet	82.23%	99.89%	90.20%	ramdo	98.20%	100.00%	99.09%
flubot	87.22%	94.92%	90.91%	ramnit	53.30%	52.45%	52.87%
fobber	30.04%	12.03%	17.18%	ranbyus	76.93%	87.64%	81.94%
gameover	99.98%	99.91%	99.95%	rovnix	94.95%	98.73%	96.80%
gameover_p2p	73.00%	60.18%	65.97%	shifu	62.12%	98.41%	76.16%
gozi	81.76%	73.31%	77.30%	simda	99.14%	99.48%	99.31%
grandoreiro	83.68%	100.00%	91.11%	sphinx	23.05%	91.38%	36.82%
locky	69.42%	46.91%	55.98%	symmi	98.50%	99.92%	99.20%
m0yvtdd	47.65%	85.66%	61.24%	tinba	95.05%	95.27%	95.16%
metastealer	99.95%	100.00%	99.98%	tinyfluff	99.80%	99.97%	99.88%
monerominer	99.99%	99.99%	99.99%	tinynuke	99.84%	100.00%	99.92%
murofet	80.51%	84.46%	82.44%	tufik	0.00%	0.00%	0.00%
murofetweekly	99.89%	99.57%	99.73%	urlzone	98.87%	90.76%	94.64%
mydoom	86.03%	95.90%	90.70%	vawtrak	80.41%	83.40%	81.87%
necro	99.37%	99.51%	99.44%	virut	99.64%	99.95%	99.80%
necurs	98.25%	80.01%	88.20%	xxhex	99.70%	100.00%	99.85%

Table A14. *Cont.*

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
ngioweb	96.67%	89.19%	92.78%	zloader	87.73%	98.45%	92.78%
nymaim	40.26%	30.90%	34.96%	suppobox	58.47%	99.94%	73.78%
nymaim2	63.54%	74.60%	68.63%	Average	82.00%	84.88%	81.51%

Table A15. Model 1, not pre-trained, trained with the same number of few examples used in the incremental method for Suppobox class, precision, recall, and F1 score of each DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	98.14%	100.00%	99.06%	orchard	99.02%	99.75%	99.39%
banjori	96.33%	99.08%	97.69%	padcrypt	47.93%	82.78%	60.71%
bazarloader	99.36%	99.26%	99.31%	phorpiex	11.55%	81.42%	20.23%
chinad	80.46%	94.93%	87.10%	pitou	80.25%	97.15%	87.90%
conficker	31.33%	26.66%	28.81%	proslikefan	6.86%	4.34%	5.32%
corebot	91.13%	93.47%	92.28%	pseudoman	11.21%	64.24%	19.08%
cryptolocker	13.63%	26.55%	18.01%	pushdo	58.86%	75.51%	66.15%
darkshell	82.65%	100.00%	90.50%	pushdotid	38.82%	71.93%	50.42%
darkwatchman	96.36%	100.00%	98.15%	pykspa2s	80.59%	83.82%	82.17%
dircrypt	1.29%	12.00%	2.33%	qadars	53.58%	91.52%	67.59%
dyre	99.93%	100.00%	99.97%	qakbot	43.40%	15.26%	22.58%
ebury	49.39%	99.82%	66.08%	qsnatch	92.84%	92.31%	92.58%
emotet	23.85%	89.04%	37.62%	ramdo	45.61%	98.06%	62.26%
flubot	63.35%	45.57%	53.01%	ramnit	15.71%	24.62%	19.18%
fobber	1.66%	1.32%	1.47%	ranbyus	38.98%	48.41%	43.18%
gameover	99.87%	99.39%	99.63%	rovnix	46.41%	94.75%	62.30%
gameover_p2p	16.71%	68.68%	26.88%	shifu	24.65%	94.93%	39.14%
gozi	14.95%	66.92%	24.43%	simda	87.62%	75.53%	81.13%
grandoreiro	89.97%	100.00%	94.72%	sphinx	3.98%	60.00%	7.46%
locky	25.21%	36.17%	29.71%	symmi	88.19%	98.63%	93.12%
m0yvtdd	41.59%	85.18%	55.89%	tinba	70.21%	39.70%	50.72%
metastealer	97.30%	99.22%	98.25%	tinyfluff	98.62%	99.87%	99.24%
moneropminer	99.42%	99.61%	99.52%	tinynuke	95.01%	100.00%	97.44%
murofet	62.67%	49.56%	55.35%	tufik	0.00%	0.00%	0.00%
murofetweekly	100.00%	97.54%	98.75%	urlzone	82.64%	69.15%	75.30%
mydoom	11.35%	87.16%	20.08%	vawtrak	22.42%	52.29%	31.39%
necro	87.12%	93.83%	90.35%	virut	99.07%	92.84%	95.85%
necurs	88.90%	45.62%	60.30%	xxhex	97.94%	100.00%	98.96%
ngioweb	63.80%	74.18%	68.60%	zloader	53.96%	79.67%	64.34%
nymaim	9.66%	30.32%	14.65%	suppobox	36.43%	100.00%	53.41%
nymaim2	16.89%	57.60%	26.12%	Average	57.16%	73.23%	60.38%

Table A16. Class Incremental Method, initial 60 DGA families without Darkshell precision, recall, and F1 score of each DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	99.79%	100.00%	99.89%	padcrypt	98.66%	98.53%	98.59%
banjori	99.91%	100.00%	99.95%	phorpiex	92.74%	80.23%	86.03%
bazarloader	99.99%	100.00%	100.00%	pitou	98.82%	97.90%	98.36%
chinad	99.40%	99.16%	99.28%	proslikefan	83.92%	19.29%	31.37%
conficker	55.40%	29.46%	38.47%	pseudoman	71.43%	77.66%	74.42%
corebot	99.83%	99.09%	99.46%	pushdo	95.93%	96.96%	96.44%
cryptolocker	57.38%	45.89%	50.99%	pushdotid	91.80%	97.92%	94.76%
darkwatchman	99.87%	99.83%	99.85%	pykspa2s	97.43%	99.21%	98.31%
dircrypt	0.00%	0.00%	0.00%	qadars	99.27%	97.29%	98.27%
dyre	99.98%	100.00%	99.99%	qakbot	72.25%	73.88%	73.05%
ebury	99.49%	97.81%	98.64%	qsnatch	98.43%	98.32%	98.37%

Table A16. *Cont.*

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
emotet	86.21%	98.67%	92.02%	ramdo	99.72%	99.94%	99.83%
flubot	91.16%	94.52%	92.81%	ramnit	55.91%	58.39%	57.12%
fobber	47.72%	16.61%	24.64%	ranbyus	82.29%	93.67%	87.62%
gameover	99.98%	99.96%	99.97%	rovnix	99.30%	98.43%	98.86%
gameover_p2p	96.43%	50.50%	66.28%	shifu	77.63%	96.79%	86.16%
gozi	85.68%	69.57%	76.79%	simda	99.22%	99.62%	99.42%
grandoreiro	99.10%	100.00%	99.55%	sphinx	50.50%	17.47%	25.95%
locky	80.80%	42.37%	55.59%	suppobox	97.45%	99.35%	98.39%
m0yvtdd	50.16%	80.05%	61.68%	symmi	100.00%	99.23%	99.61%
metastealer	99.97%	100.00%	99.99%	tinba	94.54%	99.40%	96.91%
monerominer	99.94%	100.00%	99.97%	tinyfluff	99.64%	99.83%	99.74%
murofet	76.86%	96.44%	85.54%	tinynuke	100.00%	100.00%	100.00%
murofetweekly	100.00%	99.33%	99.66%	tufik	0.00%	0.00%	0.00%
mydoom	92.95%	79.23%	85.55%	urlzone	96.45%	92.36%	94.36%
necro	99.78%	98.37%	99.07%	vawtrak	89.16%	71.39%	79.29%
necurs	94.43%	88.61%	91.43%	virut	99.63%	99.99%	99.81%
ngioweb	97.35%	81.13%	88.50%	xxhex	99.19%	100.00%	99.59%
nymaim	53.89%	14.63%	23.01%	zloader	95.24%	99.96%	97.55%
nymaim2	91.96%	56.86%	70.27%				
orchard	99.12%	99.12%	99.12%	Average	86.52%	81.67%	82.77%

Table A17. Class Incremental Method, 1st incremental train with Darkshell examples precision, recall, and F1 score of each DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	99.79%	100.00%	99.89%	padcrypt	97.73%	98.12%	97.93%
banjori	99.78%	99.91%	99.84%	phorpiex	89.45%	86.74%	88.08%
bazarloader	99.96%	100.00%	99.98%	pitou	95.68%	98.14%	96.89%
chinad	98.61%	98.85%	98.73%	proslikefan	33.27%	30.06%	31.59%
conficker	47.87%	31.26%	37.82%	pseudoman	38.81%	98.52%	55.69%
corebot	99.20%	98.87%	99.04%	pushdo	96.89%	93.85%	95.35%
cryptolocker	44.20%	60.71%	51.16%	pushdotid	88.65%	96.41%	92.37%
darkwatchman	99.74%	99.78%	99.76%	pykspa2s	97.54%	97.41%	97.48%
dircrypt	5.54%	28.67%	9.28%	qadars	96.63%	98.34%	97.48%
dyre	99.98%	100.00%	99.99%	qakbot	79.25%	63.82%	70.71%
ebury	93.96%	99.66%	96.73%	qsnatch	99.56%	96.07%	97.78%
emotet	68.44%	100.00%	81.26%	ramdo	98.25%	99.94%	99.09%
flubot	89.74%	95.51%	92.54%	ramnit	57.06%	50.55%	53.61%
fobber	25.00%	40.28%	30.85%	ranbyus	76.91%	91.57%	83.60%
gameover	99.98%	99.95%	99.96%	rovnix	95.14%	98.78%	96.93%
gameover_p2p	72.00%	57.43%	63.89%	shifu	61.24%	97.81%	75.32%
gozi	76.64%	76.94%	76.79%	simda	98.65%	97.07%	97.85%
grandoreiro	95.09%	99.40%	97.19%	sphinx	29.48%	88.18%	44.19%
locky	70.31%	47.76%	56.88%	suppobox	96.77%	98.44%	97.60%
m0yvtdd	41.67%	98.45%	58.55%	symmi	99.39%	99.54%	99.46%
metastealer	99.99%	100.00%	99.99%	tinba	94.20%	99.19%	96.63%
monerominer	99.87%	100.00%	99.93%	tinyfluff	99.47%	99.77%	99.62%
murofet	81.04%	51.95%	63.32%	tinynuke	99.30%	100.00%	99.65%
murofetweekly	99.89%	99.66%	99.78%	tufik	10.81%	40.72%	17.08%
mydoom	77.32%	93.17%	84.51%	urlzone	99.51%	91.27%	95.21%
necro	99.78%	97.73%	98.75%	vawtrak	70.26%	73.35%	71.77%
necurs	99.00%	80.37%	88.72%	virut	99.66%	99.99%	99.82%
ngioweb	96.15%	80.02%	87.35%	xxhex	98.90%	100.00%	99.45%
nymaim	33.38%	26.06%	29.27%	zloader	89.57%	99.96%	94.48%
nymaim2	86.38%	55.47%	67.55%	darkshell	49.16%	93.33%	64.40%
orchard	94.56%	99.12%	96.79%	Average	80.85%	85.15%	81.53%

Table A18. Class Incremental Method, 2nd incremental train with Darkshell examples precision, recall, and F1 score of each DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	100.00%	100.00%	100.00%	padcrypt	97.23%	98.79%	98.01%
banjori	99.55%	99.91%	99.73%	phorpiex	85.19%	90.93%	87.96%
bazarloader	99.86%	100.00%	99.93%	pitou	92.14%	98.37%	95.15%
chinad	97.74%	98.96%	98.35%	proslikefan	48.65%	26.05%	33.93%
conficker	47.18%	37.17%	41.59%	pseudoman	53.31%	96.60%	68.70%
corebot	99.66%	99.83%	99.75%	pushdo	94.22%	94.49%	94.35%
cryptolocker	42.32%	61.33%	50.08%	pushdotid	90.11%	97.59%	93.70%
darkwatchman	99.52%	99.87%	99.70%	pykspa2s	97.86%	97.58%	97.72%
dircrypt	5.25%	41.74%	9.32%	qadars	94.50%	98.84%	96.62%
dyre	99.91%	100.00%	99.96%	qakbot	77.69%	63.97%	70.16%
ebury	90.80%	99.83%	95.10%	qsnatch	99.62%	96.60%	98.09%
emotet	71.77%	100.00%	83.57%	ramdo	98.03%	99.94%	98.98%
flubot	89.73%	94.14%	91.89%	ramnit	53.96%	49.16%	51.45%
fobber	32.91%	40.81%	36.44%	ranbyus	77.39%	89.90%	83.18%
gameover	99.98%	99.91%	99.94%	rovnix	95.13%	98.61%	96.84%
gameover_p2p	73.18%	58.53%	65.04%	shifu	72.61%	96.35%	82.81%
gozi	76.08%	81.98%	78.92%	simda	98.65%	99.66%	99.15%
grandoreiro	92.98%	100.00%	96.36%	sphinx	32.51%	80.82%	46.37%
locky	64.63%	47.69%	54.88%	suppobox	94.75%	99.12%	96.89%
m0yvtdd	43.70%	98.45%	60.53%	symmi	98.77%	99.00%	98.89%
metastealer	99.99%	100.00%	100.00%	tinba	95.03%	96.67%	95.84%
monerominer	99.89%	100.00%	99.94%	tinyfluff	99.66%	99.78%	99.72%
murofet	80.27%	47.94%	60.03%	tinynuke	99.30%	100.00%	99.65%
murofetweekly	99.55%	99.55%	99.55%	tufik	11.27%	44.85%	18.02%
mydoom	63.41%	99.45%	77.45%	urlzone	99.54%	90.51%	94.81%
necro	99.64%	98.02%	98.82%	vawtrak	70.56%	77.63%	73.92%
necurs	97.22%	82.26%	89.12%	virut	99.70%	99.96%	99.83%
ngioweb	92.73%	90.07%	91.38%	xxhex	98.90%	100.00%	99.45%
nymaim	42.37%	23.66%	30.36%	zloader	92.12%	100.00%	95.90%
nymaim2	83.64%	63.02%	71.88%	darkshell	84.45%	100.00%	91.57%
orchard	86.59%	99.12%	92.43%	Average	81.56%	85.98%	82.45%

Table A19. Model 1, not pre-trained, trained with the same number of few examples used in the incremental method for Darkshell class, precision, recall, and F1 score of each DGA family.

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
bamital	97.13%	100.00%	98.54%	padcrypt	73.34%	97.72%	83.79%
banjori	88.24%	99.22%	93.40%	phorpiex	41.93%	83.95%	55.93%
bazarloader	99.93%	99.99%	99.96%	pitou	67.26%	97.67%	79.66%
chinad	82.06%	90.32%	85.99%	proslikefan	22.78%	21.86%	22.31%
conficker	23.05%	30.27%	26.17%	pseudoman	34.01%	79.88%	47.70%
corebot	82.11%	89.41%	85.61%	pushdo	60.87%	77.89%	68.33%
cryptolocker	22.50%	26.00%	24.13%	pushdotid	45.67%	72.46%	56.03%
darkwatchman	99.09%	99.78%	99.44%	pykspa2s	84.06%	82.93%	83.49%
dircrypt	1.10%	25.92%	2.11%	qadars	53.53%	84.45%	65.52%
dyre	99.96%	99.82%	99.89%	qakbot	71.12%	41.28%	52.24%
ebury	94.24%	99.33%	96.72%	qsnatch	97.83%	88.47%	92.91%
emotet	65.37%	95.44%	77.60%	ramdo	53.70%	97.99%	69.38%
flubot	75.10%	53.18%	62.27%	ramnit	12.71%	20.15%	15.59%
fobber	10.75%	39.40%	16.89%	ranbyus	45.02%	50.63%	47.66%
gameover	99.93%	98.55%	99.23%	rovnix	55.93%	92.70%	69.76%
gameover_p2p	32.04%	54.46%	40.34%	shifu	42.82%	54.01%	47.77%
gozi	10.12%	72.67%	17.76%	simda	82.71%	82.69%	82.70%
grandoreiro	73.72%	100.00%	84.87%	sphinx	5.69%	64.55%	10.45%
locky	20.03%	34.56%	25.36%	suppobox	52.40%	79.42%	63.14%

Table A19. *Cont.*

Family	Prec.	Rec.	F1	Family	Prec.	Rec.	F1
m0yvtdd	39.77%	77.59%	52.59%	symmi	82.43%	99.54%	90.18%
metastealer	99.28%	99.13%	99.20%	tinba	83.13%	43.94%	57.49%
monerominer	99.06%	99.73%	99.39%	tinyfluff	97.76%	99.51%	98.63%
murofet	65.13%	41.03%	50.34%	tinynuke	98.95%	99.82%	99.39%
murofetweekly	98.44%	99.10%	98.77%	tufik	9.91%	32.30%	15.17%
mydoom	31.43%	76.50%	44.55%	urlzone	76.14%	70.06%	72.97%
necro	86.03%	95.11%	90.35%	vawtrak	15.89%	70.17%	25.91%
necurs	91.60%	49.56%	64.32%	virut	99.22%	99.11%	99.17%
ngioweb	64.47%	81.13%	71.85%	xxhex	98.90%	99.93%	99.41%
nymaim	20.78%	25.20%	22.78%	zloader	68.06%	85.72%	75.87%
nymaim2	20.18%	61.23%	30.36%	darkshell	61.89%	100.00%	76.46%
orchard	95.58%	99.56%	97.53%	Average	62.03%	75.15%	65.30%

References

1. Kambourakis, G.; Anagnostopoulos, M.; Meng, W.; Zhou, P. *Botnets: Architectures, Countermeasures, and Challenges*, 1st ed.; CRC Press: Boca Raton, FL, USA, 2019. [CrossRef]
2. Shahzad, H.; Sattar, A.; Skandaraniyam, J. DGA Domain Detection using Deep Learning. In Proceedings of the 2021 IEEE 5th International Conference on Cryptography, Security and Privacy (CSP), Zhuhai, China, 8–10 January 2021; pp. 139–143. [CrossRef]
3. Wong, A.D. Detecting Domain-Generation Algorithm (DGA) Based Fully-Qualified Domain Names (FQDNs) with Shannon Entropy. *arXiv* **2023**, arXiv:2304.07943 .
4. Huang, W.; Zong, Y.; Shi, Z.; Wang, L.; Liu, P. PEPC: A Deep Parallel Convolutional Neural Network Model with Pre-trained Embeddings for DGA Detection. In Proceedings of the 2022 International Joint Conference on Neural Networks (IJCNN), Padua, Italy, 18–23 July 2022; pp. 1–8. [CrossRef]
5. Ren, F.; Jiang, Z.; Wang, X.; Liu, J. A DGA domain names detection modeling method based on integrating an attention mechanism and deep neural network. *Cybersecurity* **2020**, *3*, 4. [CrossRef]
6. Kruti, A.; Butt, U.; Sulaiman, R.B. A review of SolarWinds attack on Orion platform using persistent threat agents and techniques for gaining unauthorized access. *arXiv* **2023**, arXiv:2308.10294.
7. Patil, M.; Paramane, A.; Das, S.; Rao, U.; Rozga, P. Hybrid Algorithm for Dynamic Fault Prediction of HVDC Converter Transformer Using DGA Data. *IEEE Trans. Dielectr. Electr. Insul.* **2024**, *31*, 2128–2135. [CrossRef]
8. Xiao, L.; Xue, Y.; Wang, H.; Hu, X.; Gu, D.; Zhu, Y. Exploring fine-grained syntactic information for aspect-based sentiment classification with dual graph neural networks. *Neurocomputing* **2022**, *471*, 48–59. [CrossRef]
9. Wang, Y.; Pan, R.; Wang, Z.; Li, L. A Classification Method Based on CNN-BiLSTM for Difficult Detecting DGA Domain Name. In Proceedings of the 2023 IEEE 13th International Conference on Electronics Information and Emergency Communication (ICEIEC), Beijing, China, 14–16 July 2023; pp. 17–21. [CrossRef]
10. Devlin, J.; Chang, M.; Lee, K.; Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv* **2018**, arXiv:1810.04805.
11. Peters, M.; Neumann, M.; Iyyer, M.; Gardner, M.; Clark, C.; Lee, K.; Zettlemoyer, L. Deep contextualized word representations. *arXiv* **2018**, arXiv:1802.05365.
12. Gregório, J.; Cansian, A.; Neves, L.; Salvadeo, D. Deep Convolutional Neural Network and Character Level Embedding for DGA Detection. In Proceedings of the 26th International Conference on Enterprise Information Systems-Volume 2: ICEIS. INSTICC, Angers, France, 28–30 April 2024; SciTePress: Setúbal, Portugal, 2024; pp. 167–174. [CrossRef]
13. Majestic. Majestic Million. 2023. Available online: https://pt.majestic.com/reports/majestic-million (accessed on 16 August 2024).
14. NetLab 360. NetLab360. 2022. Available online: https://blog.netlab.360.com/ (accessed on 16 August 2024).
15. Ding, L.; Du, P.; Hou, H.; Zhang, J.; Jin, D.; Ding, S. Botnet DGA Domain Name Classification Using Transformer Network with Hybrid Embedding. *Big Data Res.* **2023**, *33*, 100395. [CrossRef]
16. Liew, S.R.C.; Law, N.F. Word encoding for word-looking DGA-based Botnet classification. In Proceedings of the 2023 Asia Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC), Taipei, Taiwan, 31 October–3 November 2023; pp. 1816–1821. [CrossRef]
17. Fan, B.; Ma, H.; Liu, Y.; Yuan, X.; Ke, W. KDTM: Multi-Stage Knowledge Distillation Transfer Model for Long-Tailed DGA Detection. *Mathematics* **2024**, *12*, 626. [CrossRef]
18. Nagarikar, A.; Dangi, R.S.; Maity, S.K.; Kuvelkar, A.; Wandhekar, S. Incremental Learning of Classification Models in Deep Learning. In Proceedings of the 6th International Conference on Advances in Artificial Intelligence, ICAAI '22, Birmingham, UK, 21–23 October 2023; p. 56–60. [CrossRef]
19. Yang, Q.; Gu, Y.; Wu, D. Survey of incremental learning. In Proceedings of the 2019 Chinese Control And Decision Conference (CCDC), Nanchang, China, 3–5 June 2019; pp. 399–404. [CrossRef]

20. Ramesh, R.; Chaudhari, P. Model Zoo: A Growing "Brain" That Learns Continually. *arXiv* **2022**, arXiv:2106.03027.
21. Llopis-Ibor, L.; Beltran-Royo, C.; Cuesta-Infante, A.; Pantrigo, J.J. Fast incremental learning by transfer learning and hierarchical sequencing. *Expert Syst. Appl.* **2023**, *212*, 118580. [CrossRef]
22. The Pandas Development Team. pandas-dev/pandas: Pandas; Zenodo: Geneva, Switzerland, 2020. [CrossRef]
23. Harris, C.R.; Millman, K.J.; van der Walt, S.J.; Gommers, R.; Virtanen, P.; Cournapeau, D.; Wieser, E.; Taylor, J.; Berg, S.; Smith, N.J.; et al. Array programming with NumPy. *Nature* **2020**, *585*, 357–362. [CrossRef] [PubMed]
24. Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; et al. Scikit-learn: Machine Learning in Python. *J. Mach. Learn. Res.* **2011**, *12*, 2825–2830.
25. Abadi, M.; Agarwal, A.; Barham, P.; Brevdo, E.; Chen, Z.; Citro, C.; Corrado, G.S.; Davis, A.; Dean, J.; Devin, M.; et al. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. 2015. Available online: https://tensorflow.org (accessed on 16 August 2024).
26. Fraunhofer FKIE. DGArchive. 2023. Available online: https://dgarchive.caad.fkie.fraunhofer.de/ (accessed on 16 August 2024).
27. Tanner, E.M.; Bornehag, C.G.; Gennings, C. Repeated holdout validation for weighted quantile sum regression. *MethodsX* **2019**, *6*, 2855–2860. [CrossRef] [PubMed]
28. Ravi, V.; Alazab, M.; Srinivasan, S.; Arunachalam, A.; Soman, K.P. Adversarial Defense: DGA-Based Botnets and DNS Homographs Detection through Integrated Deep Learning. *IEEE Trans. Eng. Manag.* **2023**, *70*, 249–266. [CrossRef]
29. Goodfellow, I.; Bengio, Y.; Courville, A. *Deep Learning*; MIT Press: Cambridge, MA, USA, 2016. Available online: http://www.deeplearningbook.org (accessed on 16 August 2024).
30. Zhang, A.; Lipton, Z.C.; Li, M.; Smola, A.J. *Dive into Deep Learning*; Cambridge University Press: Cambridge, UK, 2023. Available online: https://D2L.ai (accessed on 16 August 2024).
31. Koutsoukas, A.; Monaghan, K.J.; Li, X.; Huan, J. Deep-learning: Investigating deep neural networks hyper-parameters and comparison of performance to shallow methods for modeling bioactivity data. *J. Cheminform.* **2017**, *9*, 42. [CrossRef] [PubMed]
32. Dalli, A. Impact of Hyperparameters on Deep Learning Model for Customer Churn Prediction in Telecommunication Sector. *Math. Probl. Eng.* **2022**, *2022*, 4720539. [CrossRef]
33. Salehin, I.; Kang, D.K. A Review on Dropout Regularization Approaches for Deep Neural Networks within the Scholarly Domain. *Electronics* **2023**, *12*, 3106. [CrossRef]
34. Kingma, D.P.; Ba, J. Adam: A Method for Stochastic Optimization. *arXiv* **2017**, arXiv:1412.6980.
35. Highnam, K.; Puzio, D.; Luo, S.; Jennings, N.R. Real-Time Detection of Dictionary DGA Network Traffic using Deep Learning. *arXiv* **2020**, arXiv:2003.12805.
36. Sanh, V.; Debut, L.; Chaumond, J.; Wolf, T. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv* **2019**, arXiv:1910.01108.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.